

《计算机组成原理》

（第五讲）

厦门大学信息学院软件工程系 张海英

2026年5月

目录

- 第1章 计算机系统概论
- 第2章 数据信息的表示
- 第3章 运算方法与运算器
- 第4章 存储系统
- 第5章 指令系统**
- 第6章 中央处理器
- 第7章 指令流水线
- 第8章 总线系统
- 第9章 输入输出系统



第5章 指令系统

- 5.1 指令系统概述
- 5.2 指令格式
- 5.3 寻址方式
- 5.4 指令类型
- 5.5 指令格式设计
- 5.6 CISC和RISC
- 5.7 指令系统举例

5.1 指令系统概述

• 指令

- 计算机中的**指令**包括：高级语言指令、汇编语言指令、**机器指令**、**微指令**；其中高级语言指令、汇编语言指令属于软件层次，机器指令、微指令属于硬件层次
- 第5章指令系统中所介绍的指令是**机器指令**；**微指令**的内容将在第6章中央处理器中介绍

– 不同级别指令之间的关系（图5.1）：

- 一条高级语言指令“编译或解释”成多条机器指令（**1:m**）
- 一条汇编语言指令“汇编”成一条机器语言指令（**1:1**）
- 一条机器指令由若干条微指令实现（**n:1**）

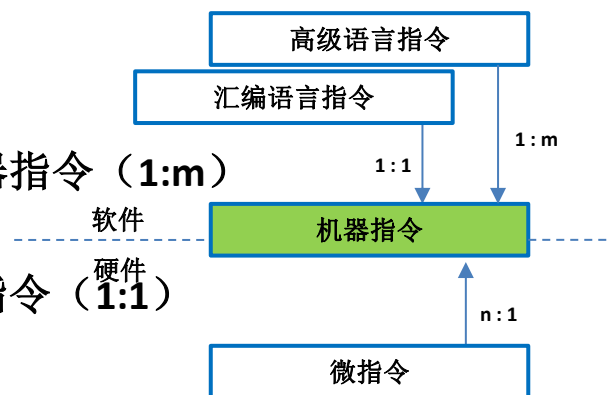


图5.1 不同级别指令之间的关系

5.1 指令系统概述

- 指令系统
 - 一台计算机中所有指令的集合称为该计算机的**指令系统**，也称**指令集**
 - 一个完善的指令系统应该满足下面的**要求**：
 - ① **完备性**：要求所设计的指令系统种类齐全、功能完备，能够编写任何可计算的程序
 - ② **规整性**：包括对称性和均齐性。**对称性**是指寄存器和存储单元都可被同等对待，所有指令都可以使用各种寻址方式。**均齐性**是指指令系统应提供不同数据类型的支持，方便程序设计，如算术运算指令能支持字节、字和双字整数运算，也能支持十进制数和单、双精度浮点运算
 - ③ **有效性**：利用指令编写的程序能高效率地运行，方便硬件实现和编译器实现，程序占用的存储资源少，运行效率高
 - ④ **兼容性**：系列计算机中新一代计算机的指令系统应该能兼容旧的指令系统（在旧计算机上开发的程序，无须修改，可以在新计算机上正确运行）
 - ⑤ **可扩展性**：指令格式中的操作码要预留一定的编码空间，以便扩展指令功能

5.2 指令格式

- 指令包括操作码OP和地址码A



图5.2 指令的一般格式

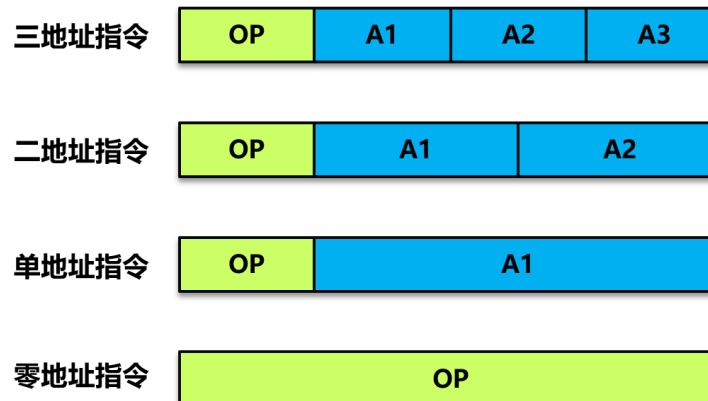
- 操作码OP (Operation code) 用于解决进行何种操作的问题
- 地址码A用于解决处理什么操作数的问题，地址码A可以包括多个操作数
- 通过何种方式获取操作数通常由寻址方式决定，寻址方式决定地址码A中操作数存放的位置和访问方式
- 寻址方式可以在地址码A中，如PDP-11指令集、Intel x86指令集；也可以在操作码OP中，如MIPS指令集、RISC-V指令集

5.2.1 指令字长度

- 指令字长度（指令长度）即指令机器码的长度，也称为指令字长
- **定长指令系统**：指令长度固定，RISC(Reduced Instruction Set Computer)计算机中多采用定长指令系统
 - 优点：结构简单，有利于CPU取指令、译码和指令顺序寻址，方便硬件执行
 - 缺点：平均指令长度较长，冗余状态较多，不容易扩展
- **变长指令系统**：指令长度可变，CISC (Complex Instruction Set Computer)计算机中多采用变长指令系统，如Intel x86序列计算机
 - 优点：结构灵活，冗余状态较少，平均指令长度较短，可扩展性好
 - 缺点：取指令和译码不方便，取指过程可能涉及多次访存操作，下一条指令的地址必须在指令译码完成后才能确定，大大增加了硬件控制系统的设计难度
- 指令字长应该是字节的整倍数
- 根据指令字长与机器字长的关系，分为：
 - ① **半字长指令**：指令字长为机器字长的一半，CPU访问一次主存可以读取两条指令
 - ② **单字长指令**：指令字长等于机器字长，CPU访问一次主存只能读取一条指令
 - ③ **多字长指令**：指令字长为机器字长的多倍，CPU需要多个存储周期才能读取一条指令
- 通常会将最常用的指令设计为短指令格式（如半字长指令、单字长指令）

5.2.2 指令地址码

- 指令中地址码A的作用随指令类型和寻址方式的不同而不同
- 地址码可能是一个操作数：MOV AH, 20H
- 也可能是操作数的地址：主存地址、寄存器编号、外部设备端口的地址等
 - MOV AH, [2000H]
 - MOV AH, AL
 - IN AL, DX
- 还可能是一个用于计算地址的偏移量：MOV AX, [SI + 5]
- 根据一条指令中所含操作数地址的数量，可将指令分为：



5.2.2 指令地址码

— 1、三地址指令：

- $A_3 \leftarrow (A_1) \text{ OP } (A_2)$ 如指令： **add rd,rs,rt** ; $R[rd]=R[rs]+R[rt]$
- 三地址指令的指令长度会很长；例如，设操作码OP=6位，存储容量=16KB，则存储器地址=14位；若三个地址均为存储器地址，则指令长度=6+14+14+14=48位
- 如果三地址指令中的操作数为寄存器，则可以缩短指令的长度；例如，设操作码OP=6位，寄存器数量=32个，则寄存器地址=5位；若三个地址均为寄存器地址，则指令长度=6+5+5+5=21位

— 2、双地址指令：

- $A_1 \leftarrow (A_1) \text{ OP } (A_2)$ 如指令： **ADD AL,BL** ; $AL \leftarrow AL+BL$
- 双地址指令有3种类型：
 - RR（寄存器-寄存器）型： **MOV AL, BL**
 - RS（寄存器-存储器）型： **MOV [2000H], AL**
 - SS（存储器-存储器）型： **MOV [2000H], [3000H]**
- RR型指令执行速度最快，使用最多；SS型指令执行速度最慢，使用最少

5.2.2 指令地址码

— 3、单地址指令：

- (1) 单目运算指令
 - $A_1 \leftarrow OP(A_1)$
 - 如指令： **INC AL** ; $AL \leftarrow AL + 1$
- (2) 隐含操作数的双目运算指令
 - $A_1 \leftarrow (AC) OP(A_1)$
 - 如指令： **MUL BL** ; AL 与 BL 相乘，结果送 AX ，隐含操作数 AX

— 4、零地址指令：

- (1) 指令本身不需要操作数
 - 如指令： **NOP** ; 空操作指令
- (2) 隐含操作数的单目运算指令
 - 如指令： **DAA** ; BCD码调整指令，将 AL 中的内容进行BCD码调整，隐含 AL

MOV AL, 43H		
MOV BL, 29H		
ADD AL, BL	;	AL=6BH
DAA	;	AL=72H 43+29=72

5.2.3 指令操作码

- 指令中操作码OP表示具体进行什么操作，如OP=0001表示进行加法操作，OP=0010表示进行减法操作
- 1、**定长操作码**
 - 操作码OP长度固定，且操作码在指令中的位置也是固定的
 - 假设指令系统有m条指令，则操作码的位数n应该满足： $n \geq \log_2 m$ ；如m=128， $\log_2 m=7$ ， $n \geq 7$
- 2、**变长操作码**
 - 操作码OP长度可变，且操作码在指令中的位置也是不固定的
 - 可以采用**扩展操作码技术**实现变长操作码，即操作码的长度随地址码数目的增加而减少；图5.3为指令字长固定的扩展操作码，指令长度=16位

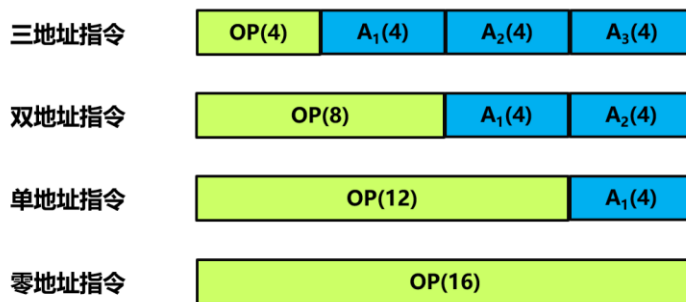


图5.3 扩展操作码

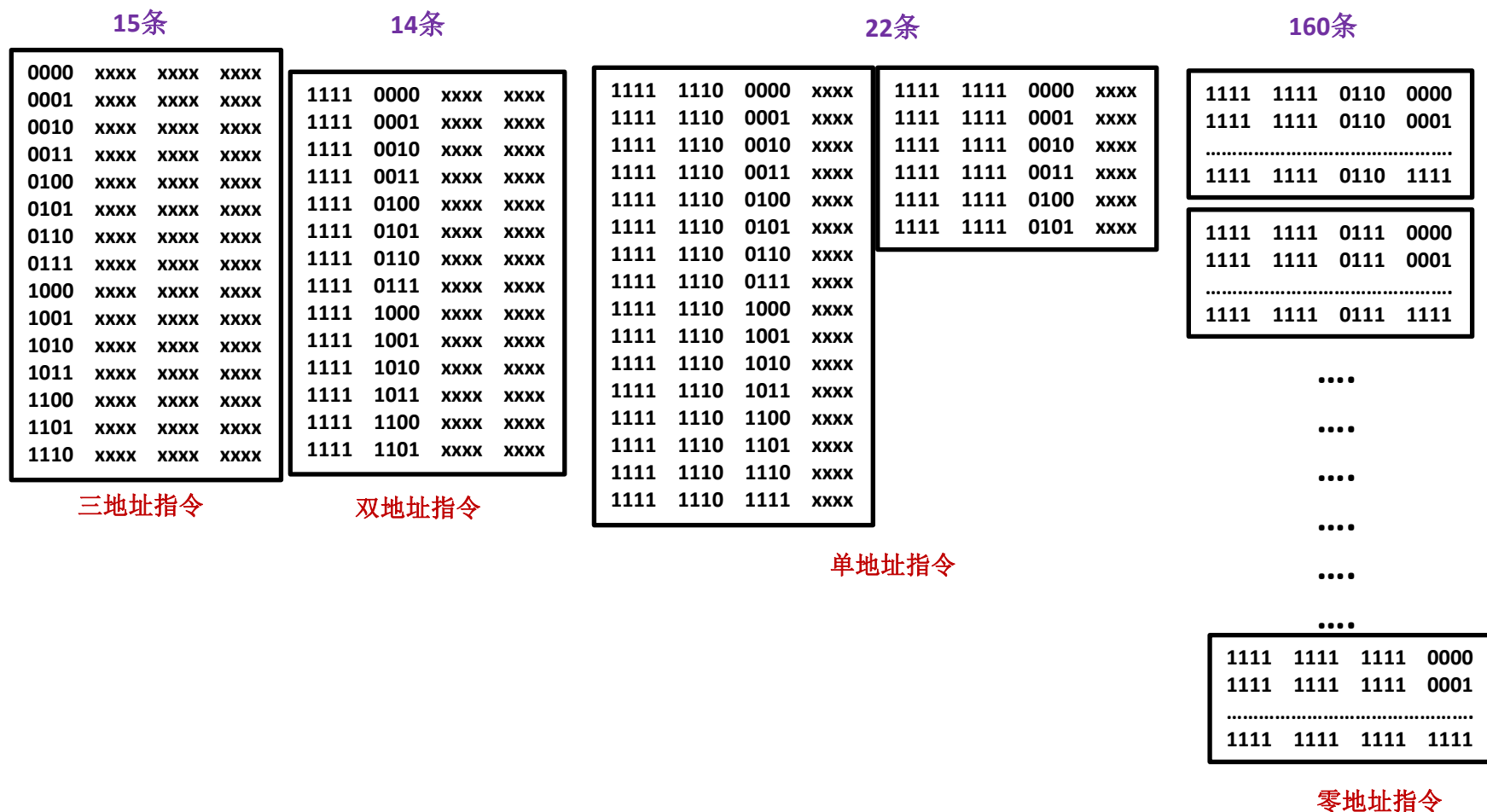
5.2.3 指令操作码

- 例5.1：假设图5.3所示的扩展操作码指令系统中有三地址指令15条、双地址指令14条、单地址指令22条，则该指令系统最多可以设计多少条零地址指令？

- 解：
 - 双地址指令只能使用三地址指令不用的剩余状态；单地址指令只能使用双地址指令不用的剩余状态；零地址指令只能使用单地址指令不用的剩余状态
 - 三地址指令的剩余状态： $2^4-15=1$
 - 双地址指令的剩余状态： $1 \times 2^4-14=2$
 - 单地址指令的剩余状态： $2 \times 2^4-22=10$
 - 零地址指令的数目： $10 \times 2^4=160$
 - 因此该指令系统最多可以设计160条零地址指令

5.2.3 指令操作码

- 例5.1：假设图5.3所示的扩展操作码指令系统中有三地址指令15条、双地址指令14条、单地址指令22条，则该指令系统最多可以设计多少条零地址指令？



5.3 寻址方式

- 指令和数据（称为操作数）是存放在主存中的，只有获得指令和操作数在主存的地址（称为**有效地址EA**， Effective Address）， CPU才能访问所需的指令和数据
- **寻址方式**就是寻找指令或操作数有效地址的方法
- 寻址方式分为**指令寻址方式**和**操作数寻址方式**

5.3.1 指令寻址方式

– 1、顺序寻址方式

- 大部分程序中的机器指令序列在主存中是按顺序存放的；因此只要知道第一条指令的有效地址，通过增加一条指令所占用主存单元的数量，就可以知道下一条指令的有效地址
- 这种计算有效地址的方法方式称为**顺序寻址方式**（图5.4a）
- **程序计数器**：PC， Program Counter，用于存放下一条指令的地址
- 假设主存按照字节进行编址；如果是单字节指令，则PC+1；如果是双字节指令，则PC+2；以此类推
- 因此，PC应具有自动加1的功能（**指令预取**）

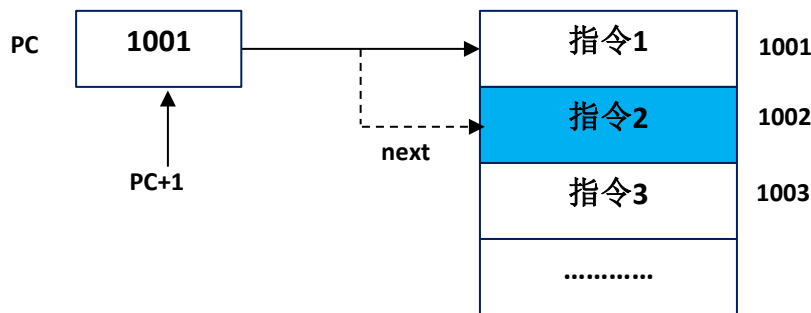


图5.4a 顺序寻址

5.3.1 指令寻址方式

– 2、跳跃寻址方式

- 如果程序中有分支或转移，就会改变程序的执行顺序，此时就需要采取**跳跃寻址方式**（图5.4b）
- 跳跃寻址方式时，下一条指令的地址由指令本身及指令需要测试的条件决定
- 无条件转移指令、条件转移指令都采用跳跃寻址方式
- 在图5.4b中，取出“**jmp 1003**”指令时，PC值先是变为1002（PC具有自动加1的功能）；之后根据“**jmp 1003**”指令，又使PC变为1003，执行完“**jmp 1003**”指令后，不是执行指令2，而是跳转到指令3执行

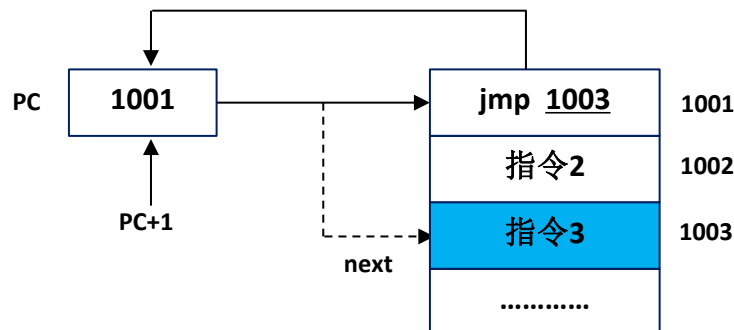


图5.4b 跳跃寻址

5.3.2 操作数寻址方式

- 操作数寻址方式就是形成操作数**有效地址**的方法
- 操作数的来源通常有3种情况：
 - ① 操作数直接来自指令地址字段：MOV AL, **23H**
 - ② 操作数存放在寄存器中：MOV AL, **BL**
 - ③ 操作数存放在存储器中：MOV AL, **[2300H]**
- 常用的操作数寻址方式有：立即寻址、直接寻址、间接寻址、寄存器寻址、寄存器间接寻址、相对寻址、变址寻址、基址寻址、堆栈寻址等
- 此时，可将指令的地址码A字段细分为**寻址方式I**字段和**形式地址D**字段；寻址方式I字段也称为寻址方式特征码

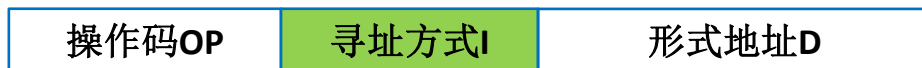


图5.5 含寻址方式字段的单地址指令格式

- 假设最终的操作数为S，有效地址为EA，则有 $S=(EA)$ ；这里()表示访问地址为EA的主存或寄存器的内容

5.3.2 操作数寻址方式

– 1、立即寻址

- 例如：MOV EAX, 2008H ； 传送类指令，其中操作数2008H采用立即寻址方式
- 图5.6a为立即寻址方式的指令格式，假设寻址方式特征码I=000
- 立即寻址方式时，形式地址D就是操作数S本省，即S=D=2008H；执行指令“MOV EAX, 2008H”后，EAX=2008H
- 立即寻址方式指令执行阶段不需要访问主存（数据不在主存，但是指令在主存）



图5.6a 立即寻址

5.3.2 操作数寻址方式

– 2、直接寻址

- 例如：MOV EAX, [2008H] ； 传送类指令，其中操作数[2008H]采用直接寻址方式
- 图5.6b为直接寻址方式的指令格式，假设寻址方式特征码=001
- 直接寻址方式时，操作数S在主存中，操作数地址（有效地址EA）由形式地址D直接给出，即有效地址EA=D=2008H，S=[2008H]
- 假设主存2008H的内容为A0A0H，执行指令“MOV EAX, [2008H]”后，EAX=A0A0H
- 直接寻址方式的特点是地址直观，指令执行阶段需要访问一次主存，但寻址范围受限于地址字段D的宽度。

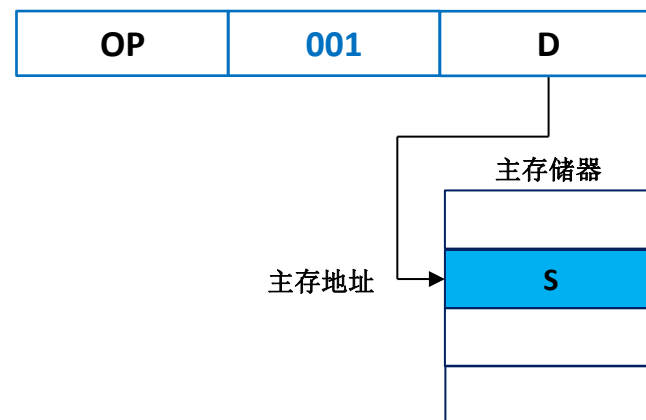


图5.6b 直接寻址

5.3.2 操作数寻址方式

– 3、寄存器寻址

- 例如：MOV **EAX**, **ECX** ； 传送类指令，其中操作数**EAX**、**ECX**采用寄存器寻址方式
- 图5.6c为寄存器寻址方式的指令格式，假设寻址方式特征码=010
- 寄存器寻址方式时，形式地址**D**表示寄存器编号，操作数**S**为寄存器中的内容，即有效地址**EA=D**，**S=R[D]**
- 寄存器寻址方式指令执行阶段不需要访问主存（所有寻址方式中**最快**）

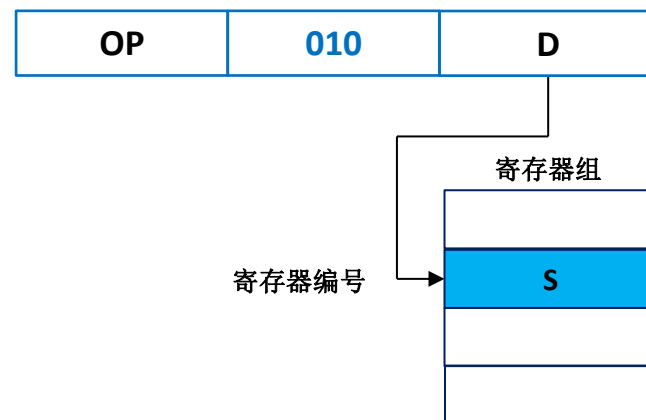


图5.6c 寄存器寻址

5.3.2 操作数寻址方式

– 4、间接寻址

- 例如：MOV EAX, @2008H ； 传送类指令，其中操作数@2008H采用间接寻址方式
- 图5.7为间接寻址方式的指令格式，假设寻址方式特征码=011

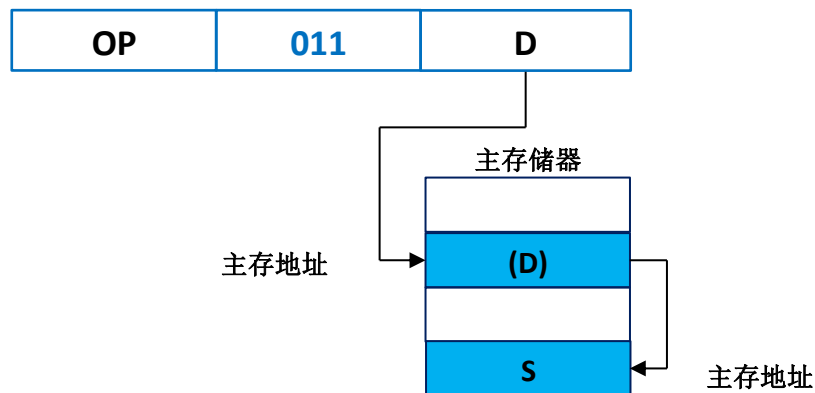


图5.7 间接寻址

- 间接寻址方式时，形式地址D给出的不是操作数的有效地址，而是操作数的间接地址；即有效地址 $EA=(D)$ ，操作数 $S=(EA)$
- 例5.2：假设主存地址2008H中的内容为A0A0F000H（32位），而主存地址A0A0F000H中的内容为5000H，则操作数 $S=5000H$ ；即执行指令“MOV EAX, @2008H”后， $EAX=5000H$
- 间接寻址方式指令执行阶段需要访问至少两次主存（1次取地址，1次取操作数，如果多次间址，则多次访存）

5.3.2 操作数寻址方式

• 5、寄存器间接寻址

- 例如：MOV AL, [EBX] ； 传送类指令，其中操作数[EBX]采用寄存器间接寻址方式

- 图5.8为寄存器间接寻址方式的指令格式，假设寻址方式特征码=100

- 寄存器间接寻址方式时，形式地址D给出的是寄存器编号，有效地址为寄存器的内容，即有效地址 $EA=R[D]$ ，操作数 $S=(EA)$

- 假设EBX=2010H，主存2010H的内容为60H，则操作数 $S=60H$ ；即执行指令“MOV AL, [EBX]”后，AL=60H

- 寄存器间接寻址方式指令执行阶段只需要访问一次主存

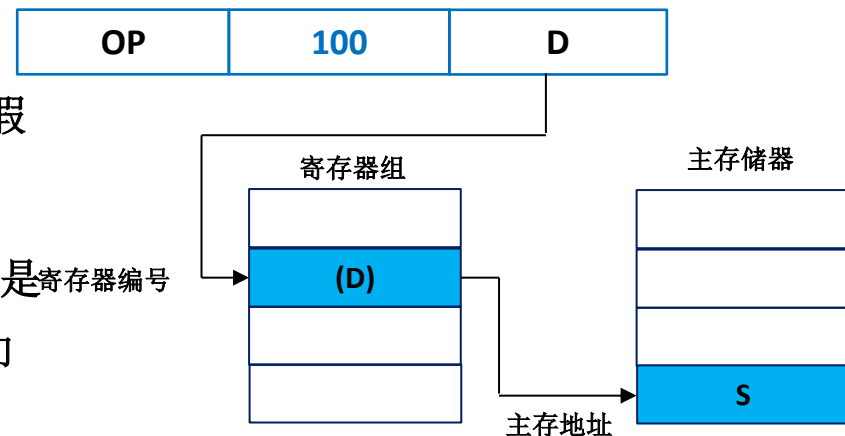


图5.8 寄存器间接寻址

5.3.2 操作数寻址方式

• 6、相对寻址

- 例如：MOV EAX, 32[PC] ； 传送类指令，其中操作数 32[PC] 采用相对寻址方式
- 图5.9为相对寻址方式的指令格式，假设寻址方式特征码 =101
- 相对寻址方式时，形式地址D给出的是偏移量，有效地址为程序计数器PC的内容加上偏移量，即有效地址 $EA=PC+D$ ；注意：这里程序计数器PC的内容为取指令完成后PC的值，需要 $PC+n \rightarrow PC$
- 假设指令“MOV EAX, 32[PC]”在取指令完成后，PC的内容为2000H，D=32H， $EA=PC+D=2000H+32H=2032H$ ，主存2032H的内容为5000H；则执行指令“MOV EAX, 32[PC]”后，EAX=5000H
- 相对寻址经常用于跳转指令，如指令“JMP LOOP”其中操作数LOOP就是采用相对寻址

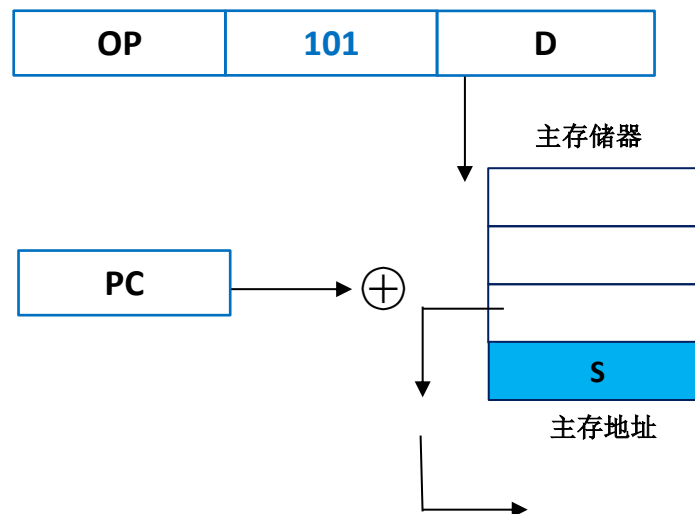


图5.9 相对寻址

- 例5.3: 某计算机指令字长为定长16位, 内存按字节寻址, 指令中的数据采用补码表示, 且PC的值在取指令阶段完成修改。请完成下列有关相对寻址的问题:
 - (1) 若采用相对寻址指令的当前地址为2003H, 且要求数据有效地址为200AH, 则该相对寻址指令的形式地址字段的值为多少?
 - (2) 若采用相对寻址转移指令的当前地址为2008H, 且要求转移后的目标地址为2001H, 则该相对寻址指令的形式地址字段的值为多少?

- 解:

- (1)
 - 有效地址EA=程序计数器PC+形式地址D, 因此 $D=EA-PC$
 - 采用相对寻址指令的当前地址为2003H, 取指令后, PC值已经指向下一条指令。因为指令字长为定长16位(2字节), 内存按字节寻址; 即: $PC\text{的值}=2003H+2H\text{ (2个字节)}=2005H$
 - 所以, $D=EA-PC=200AH-2005H=5H$; 该相对寻址指令的形式地址字段的值为5H
- (2)
 - 当前地址为2008H, 取指令后, $PC\text{的值}=2008H+16\text{位}=200AH$
 - 所以, $D=EA-PC=2001H-200AH=-9=F7H$; 该相对寻址指令的形式地址字段的值为F7H

5.3.2 操作数寻址方式

– 7、变址寻址

- 例如：MOV EAX, 32[ESI] ； 传送类指令，其中操作数32[ESI]采用变址寻址方式
- 图5.10为变址寻址方式的指令格式，假设寻址方式特征码=110，地址码字段增加一个变址寄存器X编号字段

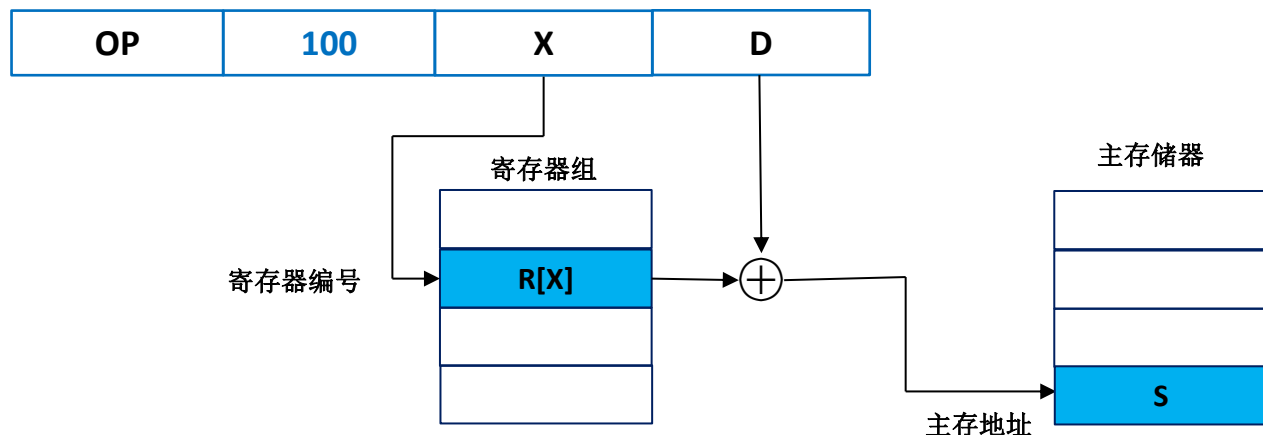


图5.10a 变址寻址

5.3.2 操作数寻址方式

– 7. 变址寻址

- 变址寻址方式时，有效地址为变址寄存器X的内容加上形式地址D，即有效地址 $EA=R[X]+D$ ；指令“MOV EAX, 32[ESI]”中，ESI为变址寄存器，32为形式地址；假设ESI的内容为2000H， $EA=2000H+32H=2032H$ ；主存2032H的内容为5000H；则执行指令“MOV EAX, 32[ESI]”后，EAX=5000H
- 变址寻址方式中，D通常是不变的，X的内容可变（用于数组寻址，D存放数组首地址，X存放数组元素的偏移量。如：A[4]，A表示首地址，4为偏移量。变址寄存器存放偏移量，通过其调整，可以访问数组元素）
- 变址寻址是面向用户的，主要用于解决程序循环问题；变址寄存器的内容由用户设置，程序执行过程中，用户通过改变变址寄存器的内容，实现指令或操作数的寻址

5.3.2 操作数寻址方式

— 8、基址寻址

- 例如：MOV EAX, 32[EBX] ； 传送类指令，其中操作数32[EBX]采用基址寻址方式
- 基址寻址方式的指令格式和变址寻址方式相同，寻址过程也与变址寻址方式相同，此时地址码字段增加一个**基址寄存器B**编号字段，如图5.10b所示

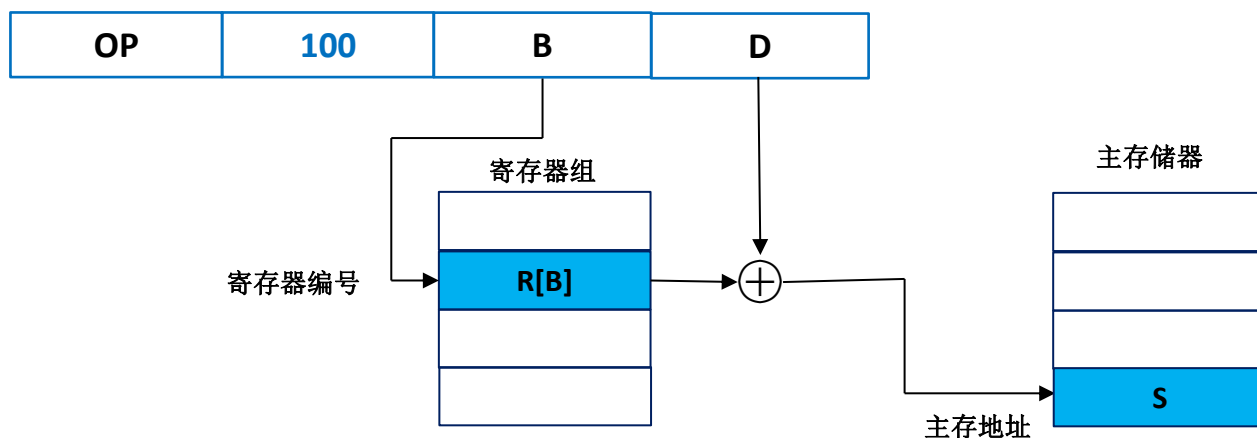


图5.10b 基址寻址

5.3.2 操作数寻址方式

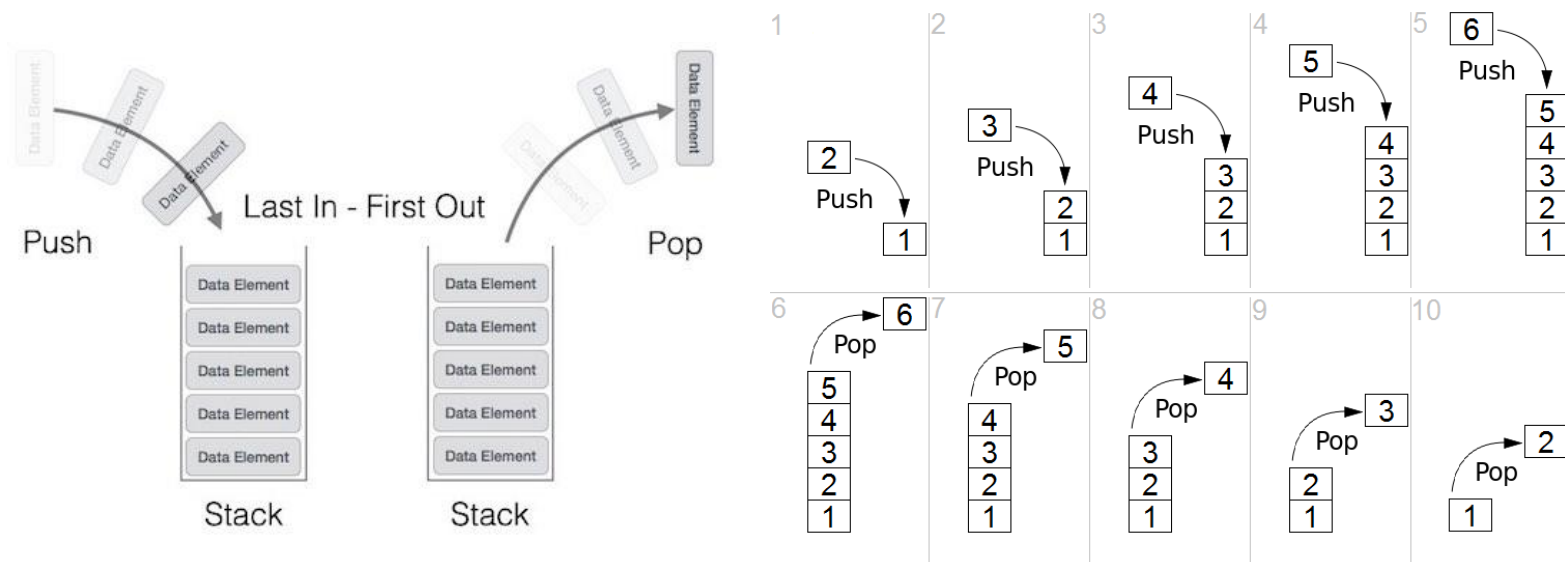
— 8.基址寻址

- 基址寻址方式中，**D**通常是可变的，**B**的内容不变；而变址寻址方式中，**D**通常是不变的，**X**的内容可变
- 基址寻址是**面向系统的**，主要用于**程序的重定位**；如第四章的**页表基址寄存器PTBR**。
- 在**多道程序**的环境下，用户不可决定自己使用的主存区，因而在编程时常按（以零为基准地址）相对地址来编写，当程序被放入主存时，操作系统根据主存空间情况给基址寄存器赋值，从而将虚地址转化为实地址。
- 相对寻址、变址寻址、基址寻址等**3**种寻址方式非常类似，也称为**偏移寻址**

5.3.2 操作数寻址方式

— 9、堆栈寻址

- **堆栈**（Stack）以先进后出（First In Last Out）的方式存储数据，堆栈通常有**存储器堆栈**和**寄存器堆栈**两种



5.3.2 操作数寻址方式

- (1) 存储器堆栈
 - 图5.11为存储器堆栈**进出栈**的操作过程，其中SP为堆栈指针寄存器（Stack Pointer）
 - 入栈操作： $SP=SP-1$ ， $M[SP]=R$ ；假设入栈数据为8位
 - 出栈操作： $M[SP]=R$ ， $SP=SP+1$ ；假设出栈数据为8位
 - 注意：如果进出栈的数据为16位，则为SP-2（SP+2）；如果进出栈的数据为32位，则为SP-4（SP+4）

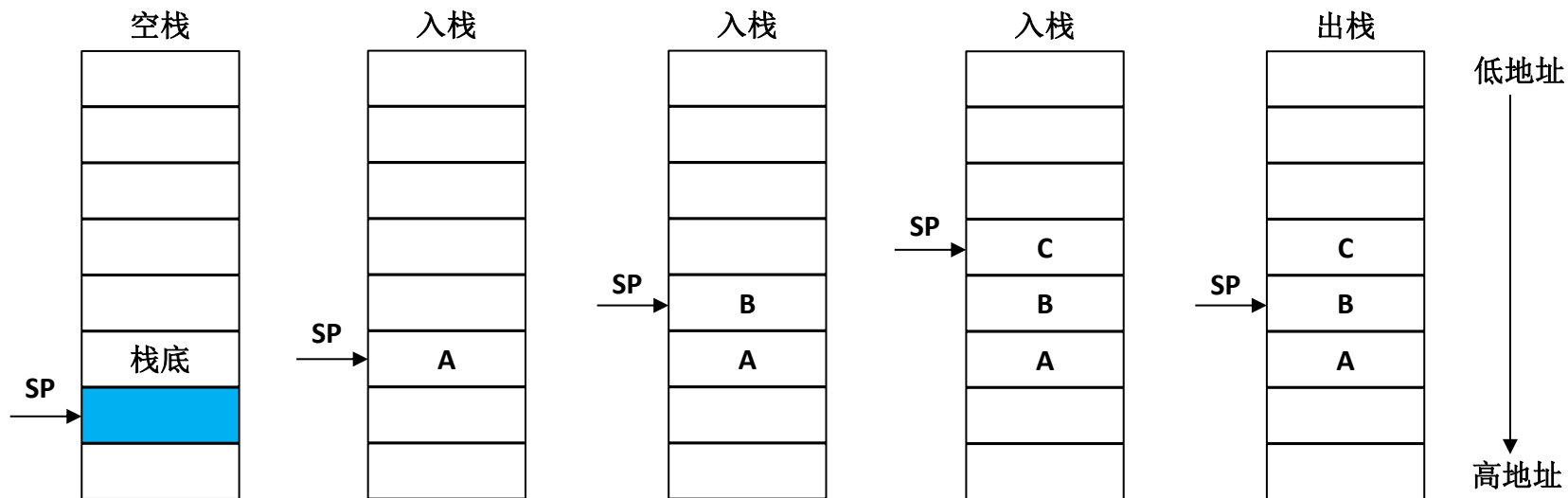


图5.11 存储器堆栈进出栈的操作过程

5.3.2 操作数寻址方式

- （2）寄存器堆栈

- 图5.12为寄存器堆栈**进出栈**的操作过程，寄存器堆栈**不需要堆栈指示器SP**

- 寄存器堆栈与存储器堆栈的不同点：

- ① 寄存器**堆栈栈顶固定不动**，而存储器堆栈栈顶随着堆栈操作而移动
- ② 进行堆栈操作时，寄存器堆栈中的**数据移动**，而存储器堆栈中的数据不动
- ③ 寄存器堆栈速度快，但**容量有限**；存储器堆栈速度慢，但容量很大
- ④ 寄存器堆栈必须采用**专用堆栈指令进行控制**，存储器堆栈则不一定

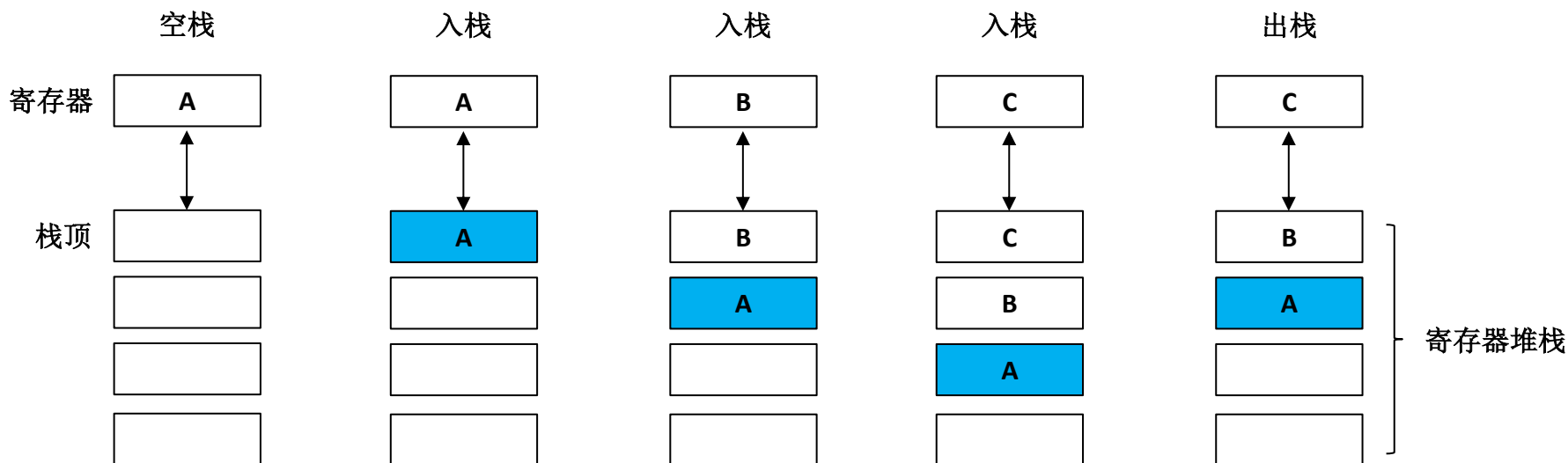


图5.12 寄存器堆栈进出栈的操作过程

5.3.2 操作数寻址方式

– 10、其他寻址

- (1) 变址 + 间接寻址方式

- 先进行变址寻址，再进行间接寻址；有效地址 $EA=R[X]+D$ ，其中 $R[X]$ 为变址寄存器 X 的内容， D 为形式地址

变址寻址: $R[X]+D$

间接寻址: $(R[X]+D)$

- (2) 间接 + 变址寻址方式

- 先进行间接寻址，再进行变址寻址；有效地址 $EA=R[X]+(D)$ ，其中 $R[X]$ 为变址寄存器 X 的内容， D 为形式地址

间接寻址: (D)

变址寻址: $R[X]+(D)$

- (3) 相对 + 间接寻址方式

- 先进行相对寻址，再进行间接寻址；有效地址 $EA=(PC+D)$ ，其中 PC 为程序计数器， D 为形式地址

相对寻址: $PC+D$

间接寻址: $(PC+D)$

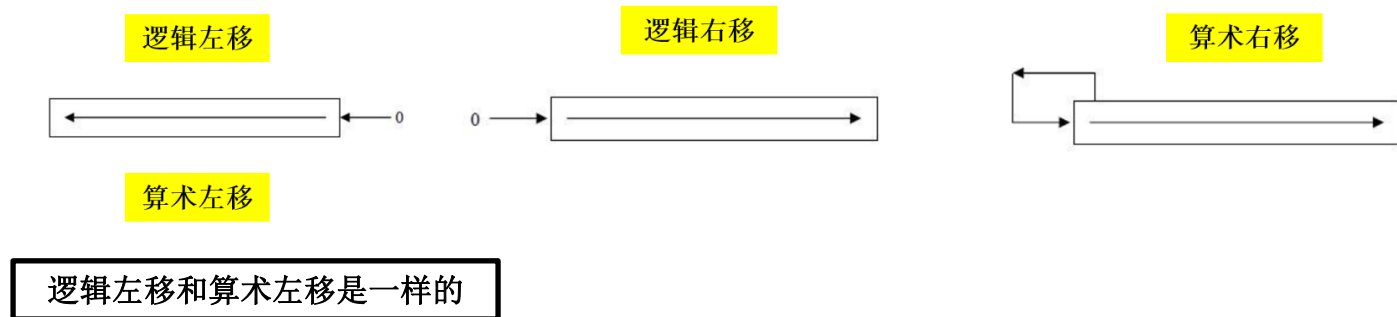
5.4 指令类型

- 指令系统决定了计算机的基本功能，不同类型的计算机，其指令系统之间的差异很大
- 有的计算机指令类型多，功能丰富，包含几百条指令；有的计算机指令类型少，功能简单，只有几十条指令
- 一个完善的指令系统应包括如下的指令类型：
- **1、算术逻辑运算指令**
 - 包括定点、浮点数的加、减、乘、除等算术运算指令，与、或、非、异或等逻辑运算指令，有些计算机还专门设置了十进制运算指令
 - 例如：**ADD AL,BL SUB AL,BL AND AL,BL XOR AL,BL**

5.4 指令类型

- 2、移位操作指令

- 算术移位指令是针对带符号数的移位操作，包括**算术左移**指令和**算术右移**指令
- 逻辑移位指令是针对无符号数的移位操作，包括**逻辑左移**指令和**逻辑右移**指令



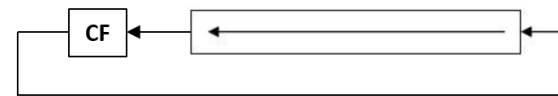
5.4 指令类型

- 2、移位操作指令

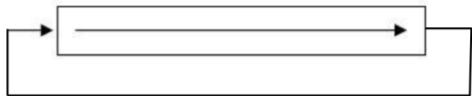
- 循环指令（小循环），包括循环左移和循环右移
- 带进位循环指令（大循环），包括带进位循环左移和带进位循环右移



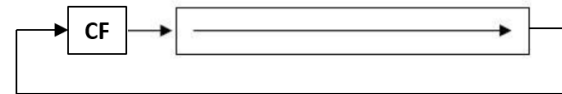
循环左移



带进位 (CF) 循环左移



循环右移



带进位 (CF) 循环右移

5.4 指令类型

- 3、数据传送指令

- 主要完成两个部件之间的数据传送，如寄存器与寄存器之间、寄存器与存储器之间、存储器与存储器之间
- 通用数据传送指令（**MOV**）：支持寄存器与寄存器之间、寄存器与存储器之间、存储器与存储器之间的数据传送
- **LOAD**（存储器读）/**STORE**（存储器写）指令：用于访问存储器，此时**MOV**指令只能用于寄存器之间的数据传送
- 交换指令（**SWAP**）：可以实现双向的数据传送
- 串传送指令（**MOVS**）：通过加上重复前缀**REP**（**REP MOVS**），可以实现将存储器中的数据块从一个区域传送到另一个区域
- 有的计算机将堆栈指令（**PUSH**、**POP**）、寄存器/存储单元清零指令（**CLR**），也归属于数据传送指令

5.4 指令类型

- 4、堆栈操作指令
 - 压栈指令（**PUSH**）：如**PUSH AX**，将寄存器**AX**的内容压入到堆栈中
 - 出栈指令（**POP**）：如**POP AX**，将堆栈中的内容弹出到寄存器**AX**中
- 5、字符串处理指令
 - 字符串处理指令属于非数值处理指令
 - 包括字符传送、字符串比较、字符串查找、字符串抽取、字符串转换等指令
 - 不是所有的计算机都支持字符串处理指令

5.4 指令类型

- 6、程序控制指令

- (1) 转移指令

- 包括：无条件转移指令（例如，JMP LOOP）和条件转移指令（例如，JZ LOOP；结果为零，则转移）

- (2) 循环控制指令

- 例如，LOOP L1，该指令每执行一次，循环计数器ECX减1，然后判断ECX是否不为0；若不为0，则转到L1处继续执行；若为0，则结束循环，执行LOOP L1指令的下一条指令

L1: MOV AL, BL
.....
.....
LOOP L1
SUB CL, DL

5.4 指令类型

- 6、程序控制指令

- (3) 子程序调用和返回指令

- 包括**CALL指令**（子程序通用指令，也称为转子指令，或过程调用指令）和**RET指令**（子程序返回指令）
 - **CALL指令**的下一条指令的地址称为**断点**；为了保证**RET指令**能够正确返回到断点，**CALL指令**应具有保护断点的功能；通常**CALL指令**会将断点压入堆栈，**RET指令**则从堆栈中取出断点，从而实现正确返回断点
 - **CALL指令**（转子指令）与转移指令的**区别**：
 - ① 转移的位置不同，转移指令在同一个程序内转移（可跨越不同CS代码段），**CALL指令**在不同程序之间转移
 - ② 转移指令不需要返回原处，而**CALL指令**需要返回原处，因此**CALL指令**需要保护断点
 - ③ **CALL指令**和**RET指令**通常是无条件的，而条件转移指令是有条件的

5.4 指令类型

- 7、输入输出指令

- 输入指令：如**IN AL, DX**；将以寄存器**DX**为端口地址的输入设备的内容送到寄存器**AL**中
- 输出指令：如**OUT DX, AL**；将寄存器**AL**的内容送到以寄存器**DX**为端口地址的输出设备中
- 当外部设备与主存采用统一编址时，则不需要设置专用的输入输出指令（**IN**和**OUT**），而是可以使用访存指令实现输入和输出指令的功能

- 8、其他指令

- 停机、等待、空操作、开中断、关中断、自陷、置条件码、特权指令等
- 特权指令一般只能用于操作系统或其他系统软件，而不直接提供给用户使用

5.5 指令格式设计

- 指令系统是程序员所能看到的计算机的主要属性，它在很大程度上决定了整个计算机系统具有的基本功能和程序性能
- 设计指令系统要**考虑**：
 - ① 指令的完备性、规整性、有效性、兼容性和可扩展性
 - ② 支持哪些指令（5.4 指令类型）
 - ③ 支持哪些数据类型和寻址方式（5.3.2 操作数寻址方式）
 - ④ 合理的指令格式（5.2 指令格式）
 - ⑤ 不仅要方便程序员进行程序设计，也有利于编译系统的设计、有利于简化硬件实现、有利于节省程序存储空间
- 指令一般由操作码和地址码组成，指令格式设计包括：指令编码格式的设计、操作码的设计、地址码的设计、寻址方式的设计

5.5 指令格式设计

- 1、指令编码格式的设计

- (1) 定长指令格式:

- 指令长度固定; 如MIPS 32指令系统, 所有的指令对应的机器码都是32位
 - 优点: 有利于简化硬件, 尤其有利于简化指令译码部件的设计
 - 缺点: 指令系统的平均长度长, 容易出现冗余码点, 不利于扩展

- (2) 变长指令格式

- 指令长度可变: 如Intel x86指令系统, 最短的指令为单字节, 最长的指令为15个字节
 - 优点: 指令平均长度短, 指令码点冗余少, 易于扩展
 - 缺点: 指令格式不规整, 不同指令的取指令时间可能不同, 控制方式较为复杂

- (3) 混合编码指令格式

- 是定长指令格式和变长指令格式的综合
 - 提供若干长度固定的指令字
 - 既能减少目标代码的长度(变长指令格式), 又能降低译码复杂度(定长指令格式)

5.5 指令格式设计

- 2、操作码的设计

- 满足完备性是操作码设计的基本要求（**完备性**：要求所设计的指令系统种类齐全、功能完备，能够编写任何可计算的程序）
- 定长结构的操作码：操作码长度固定
- 变长结构的操作码：操作码长度可变

- 3、地址码的设计

- 地址码（操作数）的数量：三地址、双地址、单地址、零地址
- 地址码设计的目的：利用有限的位宽提供更大的寻址范围

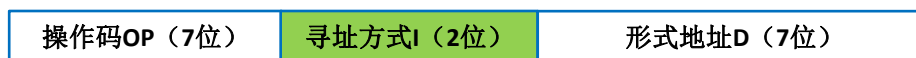
- 4、寻址方式的设计

- 一种是将寻址方式与操作码一起编码
- 另一种是设置专门的寻址方式字段（图5.5）



图5.5 含寻址方式字段的单地址指令格式

- 例5.4：某计算机字长为16位，内存为64KB，指令采用单字长、单地址结构，要求至少能支持80条指令和直接、间接、相对、变址等4种寻址方式。请设计指令格式并计算每种寻址方式能访问的主存空间范围。



• 解：

图5.13 指令格式

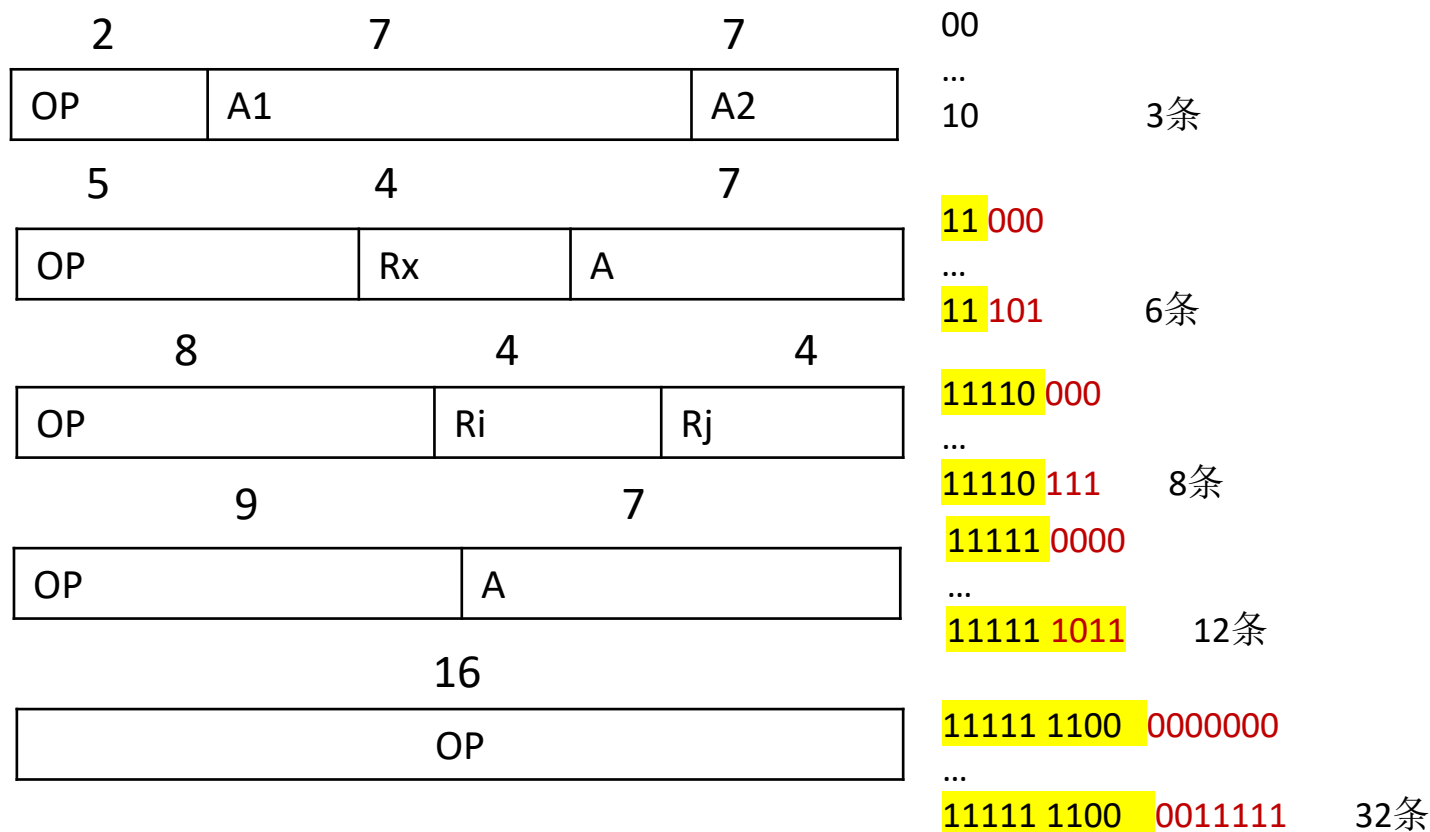
- 该指令系统为定长指令格式，指令长度=单字长=16位
- 因为至少能支持80条指令，因此操作码OP字段为7位 ($2^7 > 80$; $2^6 < 80$)
- 因为要支持4种寻址方式，因此寻址方式I字段为2位 ($2^2 = 4$)
- 因为是单地址结构，指令的格式如图5.13所示，其中形式地址D=16-7-2=7位
- 每种寻址方式的访存范围：
 - ① 直接寻址的访存空间范围：有效地址EA=D=7位，0~127
 - ② 间接寻址的访存空间范围：有效地址EA=(D)=16位，0~65535（存储器的宽度=字长=16位）
 - ③ 相对寻址的访存空间范围：有效地址EA=PC+D=16位，0~65535（程序计数器PC的长度为16位）
 - ④ 变址寻址的访存空间范围：有效地址EA=R[X]+D=16位，0~65535（变址寄存器X的长度为16位）

补充例题-唐朔飞《计算机组成原理》

- 例7.4 某机字长16位，存储器直接寻址空间128字，变址时的位移量-64~+63，16个通用寄存器均可作为变址寄存器。设计一套指令系统格式，满足下列寻址类型的要求。
 - 直接寻址的二地址指令3条
 - 变址寻址的一地址指令6条
 - 寄存器寻址的二地址指令8条
 - 直接寻址的一地址指令12条
 - 零地址指令32条
 - 试问还有多少种代码未用？若安排寄存器寻址的一地址指令，还能容纳多少条

补充例题-唐朔飞 《计算机组成原理》

- 解答：



试问还有多少种代码未用？若安排寄存器寻址的一地址指令，还能容纳多少条？

解答：还有 $2^9 - 32 = 480$ 种代码未用，若安排寄存器寻址的一地址指令，出去末4位为寄存器地址外，还可容纳30条这类指令。

补充例题-唐朔飞《计算机组成原理》

- 7.5 设某机配有基址寄存器和变址寄存器，采用一地址格式的指令系统，允许直接和间接寻址，且指令字长、机器字长和存储字长均为**16位**
 - 如采用单字长指令，并能完成**105**种操作，则指令可直接寻址的范围是多少？一次间接寻址的寻址范围是多少？画出指令格式并说明各字段含义？
 - 若存储字长不变，可采用什么方法直接访问容量为**16MB**的主存。

补充例题-唐朔飞《计算机组成原理》

- 7.5 设某机配有基址寄存器和变址寄存器，采用一地址格式的指令系统，允许直接和间接寻址，且指令字长、机器字长和存储字长均为16位
 - 如采用单字长指令，并能完成105种操作，则指令可直接寻址的范围是多少？一次间接寻址的寻址范围是多少？画出指令格式并说明各字段含义？

解答：在单字长指令中，根据能完成105种操作，取操作码7位，因允许直接和间接寻址，且有基址和变址寄存器，故取2位寻址特征位，格式如下：

OP (7)	M (2)	AD (7)
--------	-------	--------

AD代表形式地址 直接寻址范围 $2^7=128$, 间址范围 $2^{16}=65536$.

补充例题-唐朔飞《计算机组成原理》

- 7.5 设某机配有基址寄存器和变址寄存器，采用一地址格式的指令系统，允许直接和间接寻址，且指令字长、机器字长和存储字长均为**16位**
 - 若存储字长不变，可采用什么方法直接访问容量为**16MB**的主存。
 - 解答：容量**16MB**的存储器，正好与存储字长**16位**的**8M**存储器容量相等，即**16MB=8M*16位**，欲想使指令直接访问**16MB**的主存，可采用双字长指令，其操作码和寻址特征位均不变，格式如下：

OP (7)	M (2)	AD1 (7)
AD2		

5.6 CISC和RISC

• 5.6.1 复杂指令系统计算机（CISC）

- CISC: Complex Instruction Set Computer, 复杂指令系统计算机
- 计算机系统设计者在设计指令系统时, 增加了越来越多的功能强大的复杂指令, 以及更多的寻址方式, 以满足来自不同方面的需求:
 - ① 更好的支持高级语言, 增加语义接近高级语言的指令
 - ② 简化编译, 增加语义接近高级语言的指令
 - ③ 满足系列计算机软件向后兼容（向上兼容）的需求, 新的计算机指令只能增加不能减少, 指令的数量越来越多
 - ④ 对操作系统的支持, 要求指令系统提供相应功能的指令, 如多媒体指令、3D指令等
 - ⑤ 为在有限指令长度内基于扩展法实现更多指令, 只有最大限度地压缩地址码长度, 从而设计多种寻址方式

5.6.1 复杂指令系统计算机（CISC）

- 因此，计算机的指令系统越来越庞大、复杂，称为复杂指令系统计算机（CISC）
- 复杂指令系统计算机（CISC）具有如下特点：
 - ① 指令系统复杂庞大，指令数目多达200-300条
 - ② 寻址方式多
 - ③ 指令格式多
 - ④ 指令字长不固定
 - ⑤ 对访存指令不加限制
 - ⑥ 各种指令使用频率相差大
 - ⑦ 各种指令执行时间相差大
 - ⑧ 大多数采用微程序控制器

5.6.2 精简指令系统计算机（RISC）

- RISC: Reduced Instruction Set Computer, 精简指令系统计算机
- “80-20” 规律: 80%的程序只用到了20%的指令
- 精简指令系统计算机（RISC）体系结构的**基本思想**: 针对CISC指令系统指令种类太多、指令格式不规范、寻址方式太多的缺点, 通过减少指令种类、规范指令格式和简化寻址方式, 来方便处理器内部的并行处理, 从而大幅度提高处理器的性能.

5.6.2 精简指令系统计算机（RISC）

— 精简指令系统计算机（RISC）的**特点**：

- ① 优先选择**使用频率最高**的一些**简单指令**，以及一些很有用但不复杂的指令，避免使用复杂指令
- ② 大多数指令在**一个时钟周期内执行完成**
- ③ 采用**LOAD/STORE结构**，只允许LOAD/STORE指令访问主存，其余指令只能对寄存器操作数进行处理
- ④ 采用**简单的指令格式和寻址方式**，指令长度固定
- ⑤ 固定的指令格式；**指令长度、格式固定**，可简化指令的译码逻辑，有利于提高流水线的执行效率；为了便于编译的优化，常采用三地址指令格式
- ⑥ 面向寄存器的结构；为减少访问主存，**CPU**内应设大量的通用寄存器
- ⑦ 采用**硬布线控制逻辑**；由于指令系统的精简，控制部件可由组合逻辑实现，不用或少用微程序控制，这样可使控制部件的速度大大提高
- ⑧ 注重编译的优化，力求有效地**支持高级语言程序**

X86、ARM、RISC-V、MIPS

5.7 指令系统举例

- 5.7.1 PDP-11指令系统

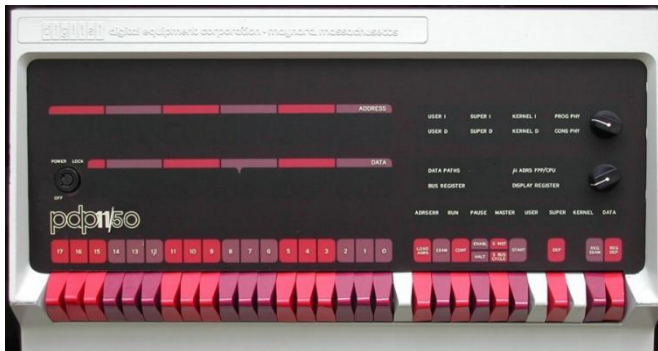
- PDP-11是DEC公司在1970年推出的一款经典的16位小型计算机，是IBM PC出现之前最为流行的小型计算机



DEC公司



- 美国DEC公司（Digital Equipment Corporation，**数字设备公司**）。1998年1月DEC公司被康柏以96亿美元的价格收购，2001年惠普康柏宣布合并。
- DEC公司于1957年诞生，1958年初，即运出了第一批产品-数字实验室和数字系统组件，第一年的销售额为9.4万美元，公司获得了一点盈利，尽管很少。1962年，是DEC公司的第五个财政年度，公司的销售额是650万美元，净利润为80万美元；1963年，DEC创利120万美元；1964年销售额上升为1100万美元，但利润下降为90万美元。DEC公司处于发展中，资金紧张，支出大于收入，1963年6月，DEC公司需要一笔现金，向ARD公司贷款30万美元，分三年还清。
- 1966年，安德森决定退出DEC公司。1965年，DEC公司的销售收入为1500万美元，1966年增长到2300万美元。
- 1966年8月16日，DEC公司公开招股上市，价格为每股22美元，ARD公司拥有175万股，占公司股份总额的65%，价值3850万美元，增值达550倍，奥尔森拥有35万股，占13%，价值770万美元；安德森拥有14万股，上市后即可转让，价值300万美元。



5.7.1 PDP-11指令系统

- PDP-11指令系统属于典型的**16位CISC指令系统**，指令字长通常为单字长（16位），有单操作数（单地址）、双操作数（双地址）两种指令格式，如图5.14a
- 双操作数时，通过采用变址寻址方式，可以增加1~2个立即数index字段，从而将指令扩展为双字长（32位）、三字长（48位）指令格式，如图5.14b

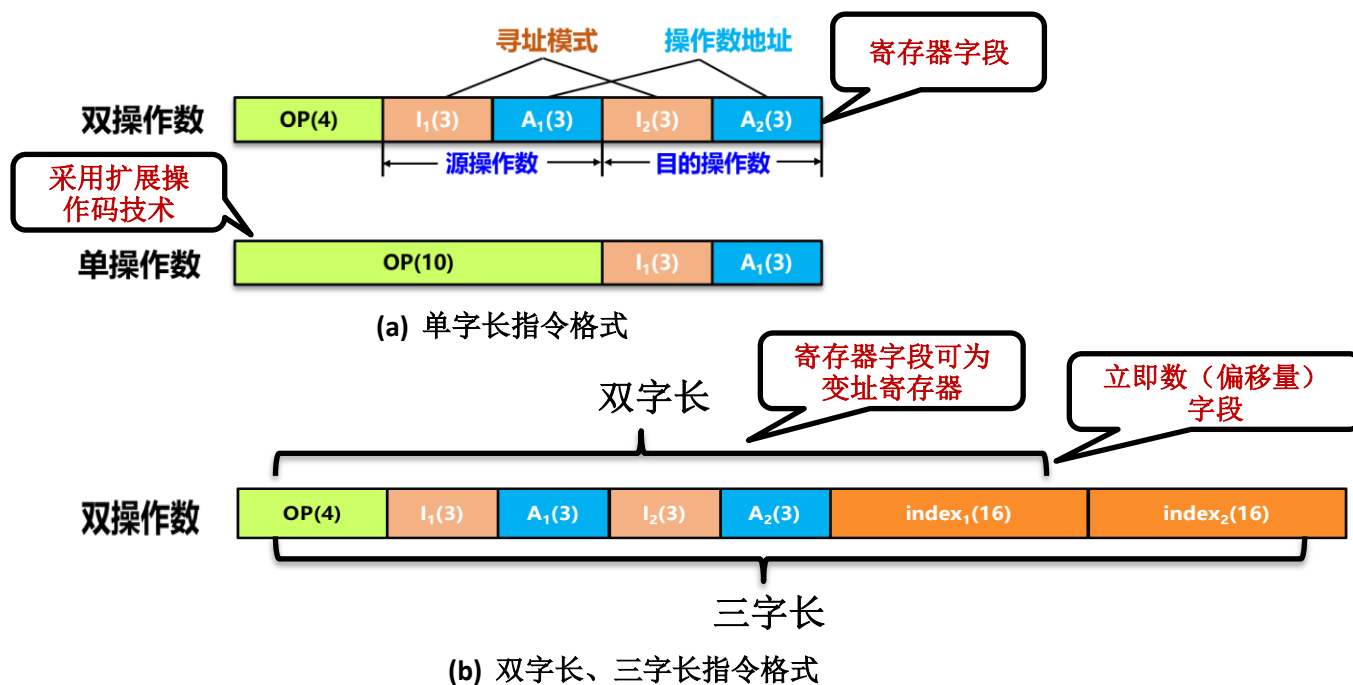


图5.14 PDP-11指令格式

5.7.1 PDP-11指令系统

- 寻址特征字段I（寻址模式）为3位，因此有8种寻址方式，如表5.1所示
- 寄存器字段A（操作数地址）为3位，可寻址8个寄存器 $R_0 \sim R_7$ ；其中 $R_0 \sim R_5$ 为通用寄存器， R_6 为SP（堆栈指针寄存器）， R_7 为PC（程序计数器）
- PDP-11的自增、自减寻址方式，方便对堆栈进行操作（SP--），也影响了C语言的语法（ $i++$ ， $++i$ ， $i--$ ， $--i$ ， $*(--i)=*(j++)$ ）

表5.1 PDP-11寻址方式

寻址模式	寻址方式	汇编语法	有效地址EA/操作数S	说明	指令实例
0	寄存器寻址	R_i	$S=(R_i)$	寄存器值就是操作数	MOV R0,R1 ;R0 <- R1
1	寄存器间接寻址	(R_i)	$EA=(R_i)$	寄存器的值是操作数地址	MOV R0,(R1) ;寄存器 <- 存储器
2	自增寻址	$(R_i)+$	$EA=(R_i), R_i++$	寄存器的值是操作数地址，取数后寄存器自增 (byte +1,word+2)	MOV R0,(SP)+ ;寄存器 <- 存储器，SP++
3	自增间接寻址	$@(R_i)+$	$EA=((R_i)), R_i++$	寄存器的值是操作数地址的地址，取数后寄存器自增 (byte +1,word+2)	INC @(R2)+ ;存储器内容自加1
4	自减寻址	$-(R_i)$	$EA=(R_i), R_i--$	先将寄存器自减 (byte -1, word -2)，运算结果是操作数地址	MOV -(SP),R0 ;存储器 <- 寄存器
5	自减间接寻址	$@-(R_i)$	$EA=((R_i)), R_i--$	先将寄存器自减 (byte -1, word -2)，运算结果是操作数地址的地址	INC @-(R2) ;存储器内容自减1
6	变址寻址	$index(R_i)$	$EA=(R_i)+index$	操作数地址=寄存器的值+16位index	ADD R0,200(R1) ;寄存器+存储器，结果送寄存器
7	变址间址寻址	$@index(R_i)$	$EA=((R_i))+index, R_i++$	操作数地址的地址=寄存器的值+16位index，寄存器自加 (byte +1, word +2)	ADD R0,@300(R2) ;寄存器+存储器，结果送寄存器

5.7.1 PDP-11指令系统

- 当寄存器字段A为111（7）时，也就是为PC寄存器时，可以扩展出4种寻址方式，如表5.2所示

表5.2 PDP-11扩展寻址方式

寻址模式	寻址方式	汇编语法	有效地址	功能	指令实例
2	立即数寻址	#n	S=index	操作数是指令字后续一个机器字	ADD #10,R0 ;R0 <- R0+10
3	直接寻址	@#n	EA=index	操作数地址是指令字后续一个机器字	CLR @#1100 ;存储器内容清零
6	相对寻址	A / A(PC)	EA=PC+index	操作数地址=PC+A A是指令字后续一个机器字	INC 10 / INC 10(PC) ;PC=PC+10; 存储器内容加1
7	相对间接	@A @A(PC)	EA=(PC+index)	操作数地址的地址=PC+A PC是下一条指令地址	CLR @10 ;Mem [PC+10]存储器内容清零

5.7.2 Intel X86指令系统

- Intel x86指令系统是由16位的8086/8088指令系统发展而来的**32位CISC指令系统**.
- 80386 CPU采用32位的Intel x86指令系统，有8个32位通用寄存器、6个16位的段寄存器，以及32位的状态寄存器和程序计数器：

① 32位通用寄存器：EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI

② 32位程序计数器（指令指针）：EIP

③ 32位状态寄存器：EFLAGS

④ 16位段寄存器：CS、DS、SS、ES、FS、GS

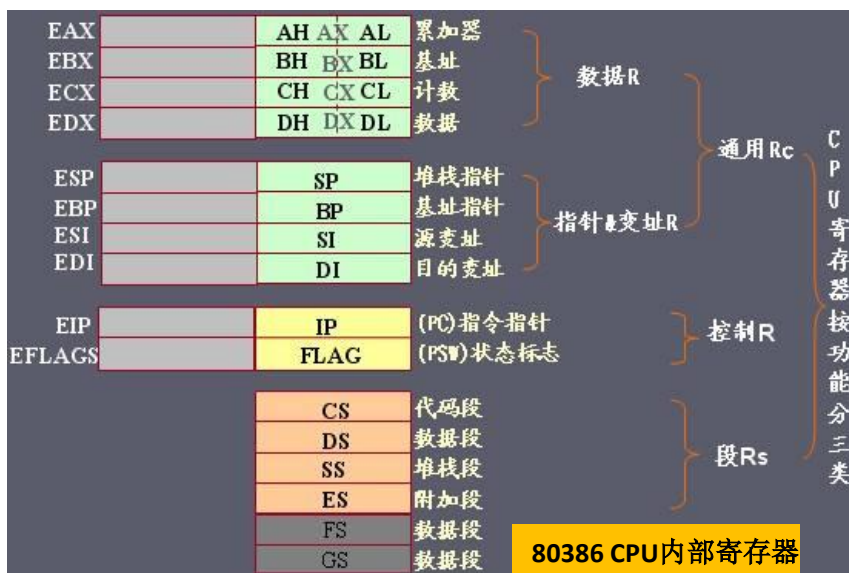


表5.3 寄存器地址编号

寄存器编号	000	001	010	011	100	101	110	111
寄存器名	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

5.7.2 Intel X86指令系统

– 1、指令格式

- 采用变长指令格式（图5.15），包括：

- ① 前缀Prefix；可选
- ② 操作码OP；必需的
- ③ Mod R/M；可选
- ④ SIB；可选
- ⑤ 偏移量Disp；可选
- ⑥ 立即数Imm；可选

- 最短的指令为单字节指令（8位），最长的指令为15字节指令（120位）

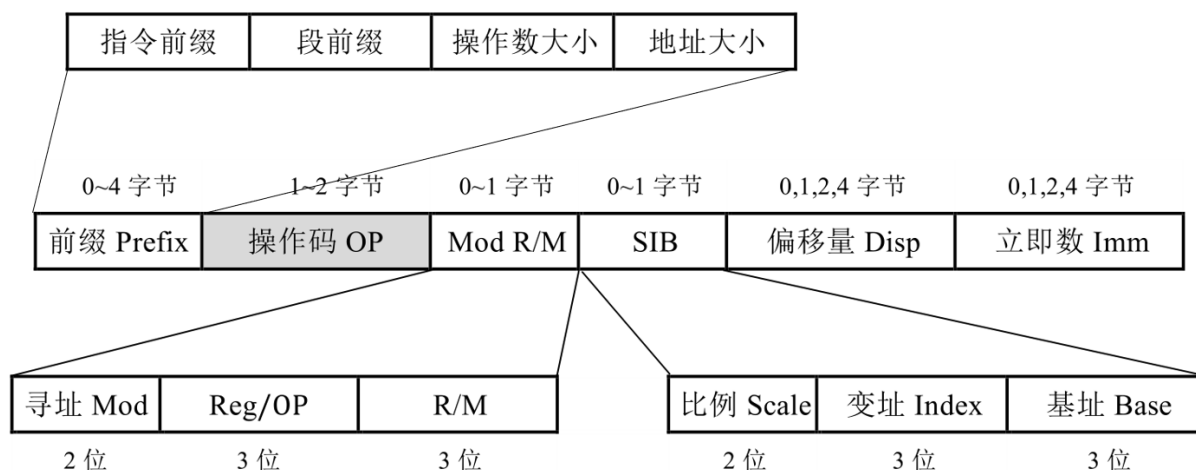


图5.15 Intel x86 指令格式

5.7.2 Intel X86指令系统

- (1) 前缀Prefix

- 前缀Prefix字段包括4部分：

- ① **指令前缀**（1个字节）：包括锁定前缀和重复前缀两种；锁定前缀（**Lock**）用于多处理器环境中共享存储器的**排他性**访问；重复前缀（**REPNE/REPZ**，**REP/REPE/REPZ**）用于进行字符串循环处理操作
 - ② **段前缀**（1个字节）：用来指定使用哪个段寄存器取代默认的段寄存器，共有6个段寄存器（**CS**、**DS**、**SS**、**ES**、**FS**、**GS**）
 - ③ **操作数大小**（1个字节）：用于在**32位**和**16位**操作数之间切换
 - ④ **地址大小**（1个字节）：用于在**32位**和**16位**地址空间之间切换，地址大小决定了指令格式中偏移量的大小和在有效地址计算中生成的偏移量的大小

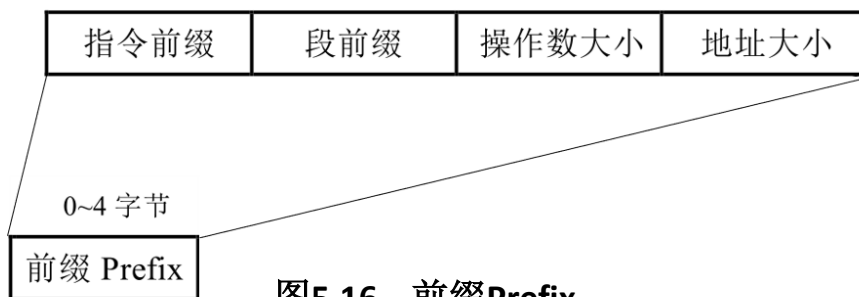


图5.16 前缀Prefix

5.7.2 Intel X86指令系统

- 不同字段的前缀值是不同的

指令前缀含义

值	含义	应用
F0H	锁定Lock前缀	用于多处理器环境中共享存储器的排他性访问
F2H	指令重复前缀REPNE/REPNZ	当ECX $\neq$ 0且两数不等时重复执行
F3H	指令重复前缀REP/REPE/REPZ	REP :当ECX $\neq$ 0重复 REPE/REPZ: 当ECX $\neq$ 0且两数相等时重复执行

段前缀含义

值	含义	值	含义
2EH	CS	26H	ES
36H	SS	64H	FS
3EH	DS	65H	GS

- 操作数大小：字段值66H，用于在32位/16位操作数之间切换
- 地址大小：字段值67H，用于在32位/16位地址空间切换

5.7.2 Intel X86指令系统

- (2) 操作码OP

- 图5.17a: 单地址指令; OP=5位; Reg=3位, 表示8个通用寄存器; 如指令push、pop、dec、inc等
- 图5.17b: 双地址指令; OP=6位; d=1, 表示Mod R/M字段中的寄存器操作数为目的寄存器, 否则为源寄存器; w=0, 表示8位操作数, 否则为16位或32位操作数 (究竟是16位还是32位由前缀Prefix决定)
- 图5.17c: OP=8位, 可以是零地址、单地址、双地址指令
- 图5.17d: OP=16位, OP的高8位为0FH、低8位为扩展操作码

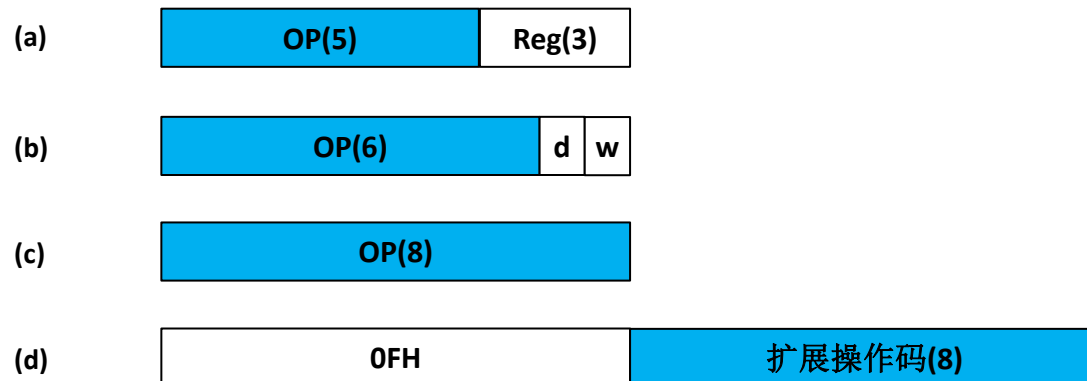


图5.17 Intel x86 指令操作码字段格式

5.7.2 Intel X86指令系统

- (3) Mod R/M

- Mod R/M字段用于描述操作数及其寻址方式，Intel x86指令集规定：双操作数最多只能有一个存储器操作数；Mod R/M字段包括3部分：

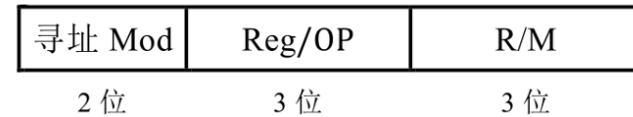


图5.18 Mod R/M字段

- 寻址Mod（2位）：与R/M字段一起构成8种寻址方式，如表5.4所示
- Reg/OP（3位）：用于表示寄存器操作数的编号（Reg）；如果是单地址指令，则为操作码的扩展字段（OP）
- R/M（3位）：用于表示另一个操作数，该操作数可能是寄存器（R），也可能是存储器（M）

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M] S在内存
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M] S在寄存器

5.7.2 Intel X86指令系统

- (4) SIB
 - SIB (**S**cale, **I**ndex, **B**ase) 字段与Mod R/M字段组合, 用于指定寻址方式; SIB字段包括3部分, 如图5.19所示:

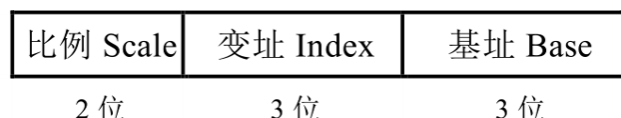


图5.19 SIB字段

- **比例Scale** (2位): 指定比例变址中的比例因子, 比例因子= 2^{Scale} (Scale=00、01、10、11, 对应1、2、4、8四个比例因子)
- **变址Index** (3位): 指定变址寄存器的编号 (8个寄存器)
- **基址Base** (3位): 指定基址寄存器的编号 (8个寄存器)
- SIB字段对应的有效地址: $\text{EA} = \text{R}[\text{Base}] + 2^{\text{Scale}} \times \text{R}[\text{Index}]$

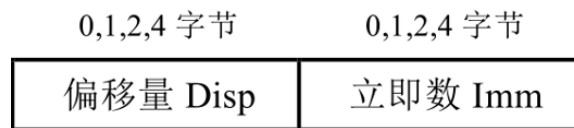
5.7.2 Intel X86指令系统

- (5) 偏移量Disp

- 偏移量Disp字段可以是8位（1字节）、16位（2字节）、32位（4字节），也可以没有（0字节）
- 偏移量字段是否存在与Mod R/M字段中的寻址方式有关

- (6) 立即数Imm字段

- 立即数Imm字段提供指令所需的8位（1字节）、16位（2字节）、32位（4字节）的立即数操作数，也可以没有（0字节）
- 立即数字段是否存在与指令操作码有关



5.7.2 Intel X86指令系统

– 2、指令译码

- 指令译码流程包括指令前缀分析、操作码译码、寻址方式译码等3步：

- ① 前缀分析：首先判断有没有前缀Prefix？如果有，则根据不同的前缀值进行指令功能控制（最多有4个字节的前缀）P171
- ② 操作码译码：首先判断是单字节操作码，还是双字节操作码（扩展操作码）？不同的操作码对应不同的指令功能；表5.5为ADD指令的部分操作码
- ③ 寻址方式译码：根据Mod R/M字段，确定指令中的操作数和寻址方式，并配合后续的SIB字段、Disp（偏移量）字段、Imm（立即数）字段，计算操作数地址

表5.5 ADD指令部分操作码				
操作码	D	W	指令形式	说明
00	0	0	ADD r/m8,r8	d=0, 寄存器为源操作数；w=0, 操作数为8位
01	0	1	ADD r/m16,r16	w=1, 操作数为16位或32位，操作数位宽取决于运行环境和指令前缀
01	0	1	ADD r/m32,r32	
02	1	0	ADD r8,r/m8	d=1, 寄存器为目的操作数
03	1	1	ADD r16,r/m16	w=1, 操作数为16/32位，操作数位宽取决于运行环境和指令前缀
03	1	1	ADD r32,r/m32	
83	1	1	ADD r/m32,imm8	源操作数为立即数Imm8; reg/op字段为操作码扩展, 为000
10000011				

目的操作数

源操作数

op(6bit)d(1bit)w(1bit)

双地址操作数，另外一个操作数寻址方式需要配合Mod R/M确定

- 例5.5：假设在一个基于Intel x86指令集的32位运行环境中，有如下指令字：
- (1) **01C2H**； (2) 2E 033BH； (3) 034C BB66H； (4) 8304 BB66H
- 请结合表5.5所示的指令操作码给出对应指令字的汇编代码。

解：

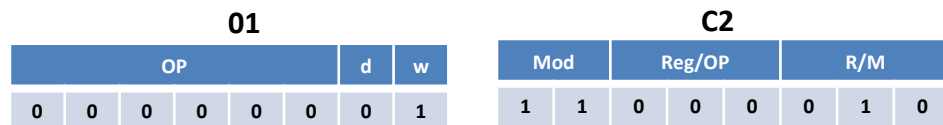


图5.20 指令字01C2H的指令格式

(1)

指令字01C2H的指令格式如图5.20所示：

操作码=01，且d=0、w=1，表示为“**ADD r/m32,r32**”指令（32位运行环境）

Mod=11，表示是寄存器寻址，因此指令为“**ADD r32,r32**”

Reg/OP=000，表示源操作数为“EAX”； R/M=010，表示目的操作数为“EDX”

d=0,表示Reg/op为源操作数

因此指令字01C2H对应的指令为“**ADD EDX, EAX**”

寄存器编号	000	001	010	011	100	101	110	111
寄存器名	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

表5.5 ADD指令部分操作码

操作码	d	w	指令形式	说明
00	0	0	ADD r/m8,r8	d=0, 寄存器为源操作数； w=0, 操作数为8位
01	0	1	ADD r/m16,r16	w=1, 操作数为16位或32位，操作数位宽取决于运行环境和指令前缀
01	0	1	ADD r/m32,r32	
02	1	0	ADD r8,r/m8	d=1, 寄存器为目的操作数
03	1	1	ADD r16,r/m16	w=1, 操作数为16/32位，操作数位宽取决于运行环境和指令前缀
03	1	1	ADD r32,r/m32	
83	1	1	ADD r/m32,imm8	Reg/OP字段为操作码扩展，值应为000

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M]
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M]

- 例5.5：假设在一个基于Intel x86指令集的32位运行环境中，有如下指令字：
- （1）01C2H；（2）2E 033BH；（3）034C BB66H；（4）8304 BB66H
- 请结合表5.5所示的指令操作码给出对应指令字的汇编代码。

解：

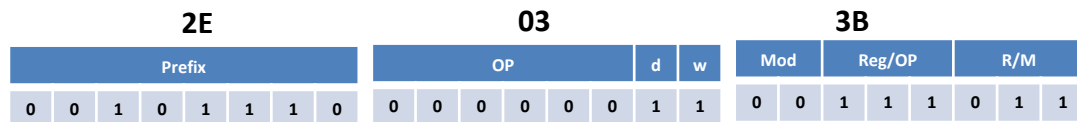


图5.21 指令字2E 033BH的指令格式

（2）

指令字2E 033BH的指令格式如图5.21所示：

寄存器编号	000	001	010	011	100	101	110	111
寄存器名	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

Prefix=2E表示段前缀，为CS段寄存器

操作码=03，且d=1、w=1，表示为“ADD r32,r/m32”指令（32位运行环境）

Mod=00，R/M=011，表示是寄存器间接寻址，因此指令为“ADD r32,m32”；因为R/M=011，表示是EBX寄存器，因此m32为CS:[EBX]（因为有段前缀）

d=1，表示Reg/op为目的操作数

Reg/OP=111，表示r32为EDI寄存器；因此指令字2E 033BH对应的指令为“ADD EDI, CS:[EBX]”

表5.5 ADD指令部分操作码

操作码	d	w	指令形式	说明
00	0	0	ADD r/m8,r8	d=0，寄存器为源操作数；w=0，操作数为8位
01	0	1	ADD r/m16,r16	w=1，操作数为16位或32位，操作数位宽取决于运行环境和指令前缀
01	0	1	ADD r/m32,r32	
02	1	0	ADD r8,r/m8	d=1，寄存器为目的操作数
03	1	1	ADD r16,r/m16	w=1，操作数为16/32位，操作数位宽取决于运行环境和指令前缀
03	1	1	ADD r32,r/m32	
83	1	1	ADD r/m32,imm8	Reg/OP字段为操作码扩展，值应为000

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M]
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M]

- 例5.5：假设在一个基于Intel x86指令集的32位运行环境中，有如下指令字：
- (1) 01C2H； (2) 2E 033BH； (3) 034C BB66H； (4) 8304 BB66H
- 请结合表5.5所示的指令操作码给出对应指令字的汇编代码。

解：

03						4C				BB						66														
OP						d	w	Mod	Reg/OP		R/M		Scale		index		Base		Disp8											
0	0	0	0	0	0	1	1	0	1	0	1	1	0	0	1	0	1	1	1	0	1	1	0	1	1	0	0	1	1	0

(3)

图5.22 指令字034C BB66H的指令格式

指令字034C BB66H的指令格式如图5.22所示：

- 操作码=03，且d=1、w=1，表示为“ADD r32,r/m32”指令（32位运行环境）
- Mod=01，R/M=100，表示是基址+比例变址+偏移量寻址，因此指令为“ADD r32,m32”
- $EA = SIB + Disp8 = R[Base] + 2^{Scale} \times R[Index] + 66$ ；Base=011，因此R[Base]=EBX；Index=111，因此R[Index]=EDI；Scale=10，因此 $2^{Scale}=4$ ；Disp8=66，因此m32为[EBX+EDI*4+66]
- Reg/OP=001，表示r32为ECX寄存器；因此指令字034C BB66H对应的指令为“ADD ECX, [EBX+EDI*4+66]”

寄存器编号	000	001	010	011	100	101	110	111
寄存器名	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

表5.5 ADD指令部分操作码

操作码	d	w	指令形式	说明
00	0	0	ADD r/m8,r8	d=0，寄存器为源操作数；w=0，操作数为8位
01	0	1	ADD r/m16,r16	w=1，操作数为16位或32位，操作数位宽取决于运行环境和指令前缀
01	0	1	ADD r/m32,r32	
02	1	0	ADD r8,r/m8	d=1，寄存器为目的操作数
03	1	1	ADD r16,r/m16	w=1，操作数为16/32位，操作数位宽取决于运行环境和指令前缀
03	1	1	ADD r32,r/m32	
83	1	1	ADD r/m32,imm8	Reg/OP字段为操作码扩展，值应为000

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M]
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M]

- 例5.5：假设在一个基于Intel x86指令集的32位运行环境中，有如下指令字：
- （1）01C2H；（2）2E 033BH；（3）034C BB66H；（4）8304 BB66H
- 请结合表5.5所示的指令操作码给出对应指令字的汇编代码。

解：

83								04						BB						66											
OP						d	w	Mod		Reg/OP			R/M			Scale		index		Base		Imm8									
1	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	1	0	1	1	1	0	1	1	0	1	1	0	0	1	1	0

- （4）**扩展操作码**
指令字8304 BB66H的指令格式如图5.23所示：

图5.23 指令字8304 BB66H的指令格式

寄存器编号	000	001	010	011	100	101	110	111
寄存器名	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

- 操作码=83，且d=1、w=1，表示为“**ADD r/m32, imm8**”指令
- Mod=00，R/M=100，表示是基址+比例变址寻址，因此指令为“**ADD r32, imm8**”
- $EA = SIB = R[Base] + 2^{Scale} \times R[Index]$ ；Base=011，因此R[Base]=EBX；Index=111，因此R[Index]=EDI；Scale=10，因此 $2^{Scale}=4$ ；因此m32为[EBX+EDI*4]
- 此时，Reg/OP=000，为扩展操作码；因此指令字8304 BB66H对应的指令为“**ADD [EBX+EDI*4], 66**”

表5.5 ADD指令部分操作码

操作码	d	w	指令形式	说明
00	0	0	ADD r/m8,r8	d=0，寄存器为源操作数；w=0，操作数为8位
01	0	1	ADD r/m16,r16	w=1，操作数为16位或32位，操作数位宽取决于运行环境和指令前缀
01	0	1	ADD r/m32,r32	
02	1	0	ADD r8,r/m8	d=1，寄存器为目的操作数
03	1	1	ADD r16,r/m16	w=1，操作数为16/32位，操作数位宽取决于运行环境和指令前缀
03	1	1	ADD r32,r/m32	
83	1	1	ADD r/m32,imm8	Reg/OP字段为操作码扩展，值应为000

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M]
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M]

5.7.2 Intel X86指令系统

– 3、寻址方式

- 操作数的主要来源为：立即数、寄存器、存储单元；当操作数为**存储单元**时，需要进行**地址转换**
- x86处理器采用**段页式**存储管理机制，有**3**类地址：逻辑地址、线性地址、物理地址
- **逻辑地址**为段式虚拟地址，包括段寄存器和段内偏移地址，即指令中的有效地址EA
- **线性地址**（LA: Linear Address）为页式虚拟地址（见4.6.3 页式虚拟存储器）
- 地址转换时，首先通过**分段方式**将逻辑地址转换为线性地址，然后再通过**分页方式**将线性地址转换为**物理地址**

逻辑地址—>线性地址—>物理地址

- 在**实模式**下：物理地址（20位）= 段寄存器（16位）左移4位 + 段内偏移地址
- 在**保护模式**下：线性地址（32位）= 段基址（32位）+ 段内偏移地址；再将线性地址通过存储器管理部件（MMU）转换为物理地址（32位）
- 地址转换中最关键的是**段内偏移地址**的计算，段内偏移地址由寻址方式确定，x86的主要寻址方式如表5.6所示

5.7.2 Intel X86指令系统

表5.6 x86的主要寻址方式

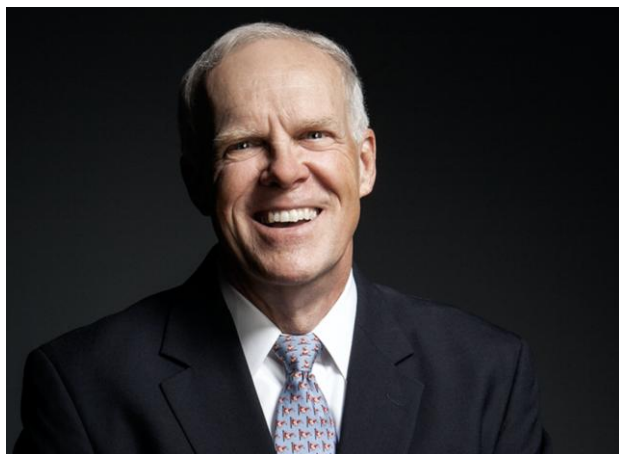
序号	寻址方式	有效地址EA/操作数S	指令示例
1	立即数寻址	$S = \text{Disp}$	MOV EAX, 1000
2	直接寻址	$EA = \text{Disp}$	MOV EAX, [1080H]
3	寄存器寻址	$S = R[R/M]$	MOV EAX, ECX
4	寄存器间接寻址	$EA = R[R/M]$	MOV EAX, [EBX]
5	基址寻址（寄存器相对寻址）	$EA = R[R/M] + \text{Disp8}$	MOV EAX, [ESI+100H]
6	基址+比例变址寻址	$EA = S * \text{index} + \text{Base}$	MOV EAX, [EBX+EDI*4]
7	基址+比例变址+偏移量寻址	$EA = S * \text{index} + \text{Base} + \text{Disp}$	MOV EAX, [EBX+EDI*4+66]
8	相对寻址	$EA = PC + \text{Disp}$	JMP 1000H

5.7.3 MIPS指令系统

- MIPS: Microprocessor without Interlocked Pipeline Stages, 无互锁流水线的微处理器
- MIPS: Million Instructions Per Second, 每秒执行百万条指令
- MIPS指令系统是1981年斯坦福大学Hennessy教授研究小组研制并商用, 是一种非常优雅、简洁、高效的RISC体系结构, 非常适合于教学研究, 包括MIPS 32 (32位架构) 和MIPS 64 (64位架构)
- 龙芯处理器就是基于MIPS指令系统

John Hennessy

- 约翰·汉尼斯（John Hennessy，1953年—），美国计算机科学家。为MIPS科技公司创办人，第十任斯坦福大学校长。
- 毕业于维拉诺瓦大学取得电机工程学士学位，毕业于纽约石溪大学取得计算机科学硕士及博士学位。
- 1984年创办MIPS科技公司。1997年成为计算机协会成员。2000年，得到约翰·冯诺依曼奖。





龙芯



- **2002年8月10日**诞生的“龙芯一号”是我国首枚拥有自主知识产权的通用高性能微处理芯片。龙芯从**2001年**以来共开发了**1号、2号、3号**三个系列处理器和龙芯桥片系列，在政企、安全、金融、能源等应用场景得到了广泛的应用。龙芯**1号**系列为**32位**低功耗、低成本处理器，主要面向低端嵌入式和专用应用领域；龙芯**2号**系列为**64位**低功耗单核或双核系列处理器，主要面向工控和终端等领域；龙芯**3号**系列为**64位**多核系列处理器，主要面向桌面和服务器等领域。
- **2015年3月31日**，中国发射首枚使用“龙芯”北斗卫星。
- **2019年12月24日**，龙芯**3A4000/3B4000**在北京发布，使用与上一代产品相同的**28nm**工艺，通过设计优化，实现了性能的成倍提升。龙芯坚持自主研发，芯片中的所有功能模块，包括**CPU**核心等在内的所有源代码均实现自主设计，所有定制模块也均为自主研发。
- **2020年3月3日**，**360公司**与龙芯中科技术有限公司联合宣布，双方将加深多维度合作，在芯片应用和网络安全开发等领域进行研发创新，并展开多方面技术与市场合作。
- **2021年4月**龙芯自主指令系统架构（**Loongson Architecture**，以下简称龙芯架构或**LoongArch**）的基础架构通过国内第三方知名知识产权评估机构的评估。
- **2021年12月17日**，龙芯中科技术股份有限公司（简称“**龙芯中科**”）首发上会。

5.7.3 MIPS指令系统

— 1、MIPS体系结构中的寄存器

- 32个32位的通用寄存器：**\$0 ~ \$31**；也可以表示为：\$zero、\$at、.....、\$ra；见表5.7
- 32个32位单精度浮点寄存器：**f0 ~ f31**；它们可配对为16个64位的浮点寄存器
- 2个特殊的寄存器：**\$hi、\$lo**，用于保存乘（64位的乘积）、除（商和余数）运算结果

表5.7 MIPS的通用寄存器

寄存器#	助记符	释义
\$0	\$zero	固定值为0 硬件置位
\$1	\$at	汇编器保留，临时变量
\$2~\$3	\$v0~\$v1	函数调用返回值
\$4~\$7	\$a0~\$a3	4个函数调用参数
\$8~\$15	\$t0~\$t7	暂存寄存器，被调用者按需保存
\$16~\$23	\$s0~\$s7	save寄存器，调用者按需保存
\$24~\$25	\$t8~\$t9	暂存寄存器，同上
\$26~\$27	\$k0~\$k1	操作系统保留，中断异常处理
\$28	\$gp	全局指针 (Global Pointer)
\$29	\$sp	堆栈指针 (Stack Pointer)
\$30	\$fp	帧指针 (Frame Pointer)
\$31	\$ra	函数返回地址 (Return Address)

静态数据访问

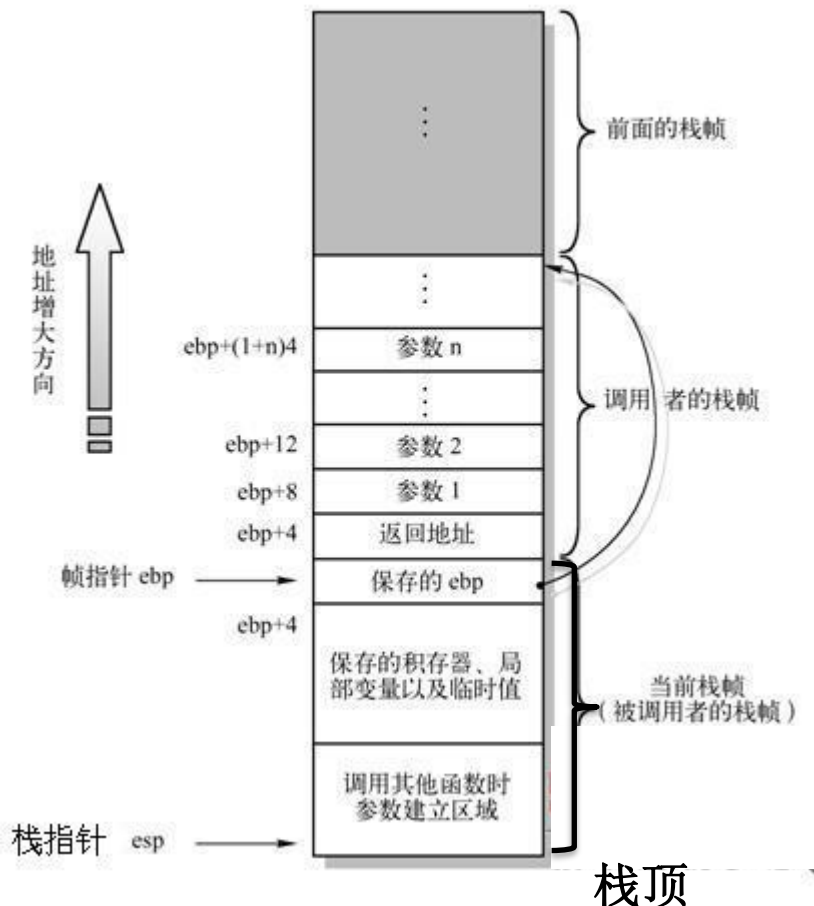
用于过程调用

子程序返回地址

关于ebp

- 栈用来传递函数参数，存储返回信息，临时保存寄存器原有值以备恢复以及用来存储局部数据。单个函数调用操作所使用的栈部分被称为栈帧（**stack frame**），结构如图所示
- 栈帧结构的两端由两个指针来指定
- 寄存器**ebp**通常用作帧指针
- **esp**用栈指针
- 在函数执行中，栈指针**esp**会随着数据的入栈和出栈移动，因此函数中大部分数据的访问都基于帧指针**ebp**进行。

[栈指针&& 帧指针详解](#)



5.7.3 MIPS指令系统

— 2、MIPS指令格式

- 所有的指令都是**32位的定长指令**（每条指令对应的机器码都是32位），包括**R型（寄存器型）、I型（立即数型）、J型（无条件转移型）**3种指令格式，见图5.24
- 操作码OP=6位；R型指令的OP=000000，采用扩展操作码的方式，**指令的功能由funct字段（6位）决定**
- **Rs、Rt、Rd**为寄存器操作数字段，均为5位，可以访问32个通用寄存器
- **shamt**字段为**移位变量**（5位），用于移位指令，其他指令无效
- 立即数有**16位（I型指令）**和**26位（J型指令）**两种
- 指令字中没有独立的寻址方式字段，**指令的寻址方式由操作码决定**

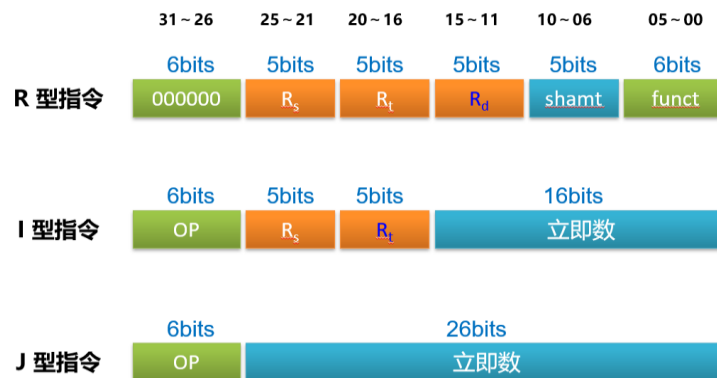


图5.24 MIPS的指令格式

- (1) R型指令（寄存器型）



- R型指令的操作数只能是寄存器，运算结果也只能存入寄存器，属于**RR（寄存器-寄存器）型**指令
- **OP=000000**，指令的功能由**funct**字段指定（6位，64种功能），如表5.8所示
- 如果是双目运算，**rs**、**rt**分别是第一和第二源操作数，**rd**为目的操作数；如果是移位运算，则表示对**rt**的内容进行移位，移位位数由**shamt**指定

表5.8 常用R型指令及其funct编码

funct	指令助记符	指令功能描述	备注
00	sll rd,rt,shamt	$R[rd]=R[rt]<<shamt$	逻辑左移，注意rs字段未使用
02	srl rd,rt,shamt	$R[rd]=R[rt]>>shamt$	逻辑右移，注意rs字段未使用
03	sra rd,rt,shamt	$R[rd]=R[rt]>>shamt$	算术右移，注意rs字段未使用
04	sliv rd,rt,rs	$R[rd]=R[rt]<<R[rs]$	可变左移
08	jr rs	$PC=R[rs]$	$R[rs]$ 值应是4的倍数，字对齐
09	jalr rs	$R[31]=PC+8$ $PC=R[rs]$	子程序调用， $R[31]$ 存放子程序返回地址 +8 有延迟槽，无延迟槽+4
12	syscall	系统调用	无操作数
16	mfhi rd	$R[rd]=HI$	取HI寄存器的值，mflo取LO
17	mfhi rs	$HI=R[rs]$	存HI寄存器的值，mflo存LO
24	mult rs,rt	$\{HI,LO\}=R[rs]*R[rt]$	有符号乘，64位结果送入HI、LO寄存器
32	add rd,rs,rt	$R[rd]=R[rs]+R[rt]$	溢出时发生异常，且不修改R[rd]
34	sub rd,rs,rt	$R[rd]=R[rs]-R[rt]$	溢出时发生异常，且不修改R[rd]
36	and rd,rs,rt	$R[rd]=R[rs]\&R[rt]$	逻辑与
37	or rd,rs,rt	$R[rd]=R[rs] R[rt]$	逻辑或
42	slt rd,rs,rt	$R[rd]=(R[rs]<R[rt])?1:0$	小于置位指令，有符号比较

- 例5.6：根据MIPS指令操作码定义及指令格式，写出下列指令各字段的十进制值：**add \$s1, \$t0, \$s4。**

解：

- add为R型指令：**add rd,rs,rt**；因此OP=000000=0，**func=32**



表5.8 常用R型指令及其func编码

func	指令助记符	指令功能描述	备注
00	sll rd,rt,shamt	$R[rd] \leftarrow R[rt] \ll \text{shamt}$	逻辑左移，注意rs字段未使用
02	srl rd,rt,shamt	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	逻辑右移，注意rs字段未使用
03	sra rd,rt,shamt	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	算术右移，注意rs字段未使用
04	slv rd,rt,rs	$R[rd] \leftarrow R[rt] \ll R[rs]$	可变左移
08	jr rs	$PC \leftarrow R[rs]$	$R[rs]$ 值应是4的倍数，字对齐
09	jalr rs	$R[31] \leftarrow PC \div 4$ $PC \leftarrow R[rs]$	子程序调用
12	syscall	系统调用	无操作数
16	mflr rd	$R[rd] \leftarrow HI$	取HI寄存器的值，mflr取LO
17	mthi rs	$HI \leftarrow R[rs]$	存HI寄存器的值，mthi存LO
24	mult rs,rt	$(HI, LO) \leftarrow R[rs] * R[rt]$	右符号乘，64位结果送入HI、LO寄存器
32	add rd,rs,rt	$R[rd] \leftarrow R[rs] + R[rt]$	溢出时发生异常，且不修改R[rd]
34	sub rd,rs,rt	$R[rd] \leftarrow R[rs] - R[rt]$	溢出时发生异常，且不修改R[rd]
36	and rd,rs,rt	$R[rd] \leftarrow R[rs] \& R[rt]$	逻辑与
37	or rd,rs,rt	$R[rd] \leftarrow R[rs] R[rt]$	逻辑或
42	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1 : 0$	小于置位指令，有符号比较

- 根据MIPS寄存器定义，**rd=\$s1=\$17**，**rs=\$t0=\$8**，**rt=\$s4=\$20**；shamt字段没有使用

- 因此，**add \$s1, \$t0, \$s4**对应的各字段的十进制值如图5.25所示

OP=0	rs=8	rt=20	rd=17	shamt=0	func=32
------	------	-------	-------	---------	---------

表5.7 MIPS的通用寄存器

寄存器#	助记符	释义
\$0	\$zero	固定值为0 硬件置位
\$1	\$at	汇编器保留，临时变量
\$2~\$3	\$v0~\$v1	函数调用返回值
\$4~\$7	\$a0~\$a3	4个函数调用参数
\$8~\$15	\$t0~\$t7	暂存寄存器，被调用者按需保存
\$16~\$23	\$s0~\$s7	save寄存器，调用者按需保存
\$24~\$25	\$t8~\$t9	暂存寄存器，同上
\$26~\$27	\$k0~\$k1	操作系统保留，中断异常处理
\$28	\$gp	全局指针 (Global Pointer)
\$29	\$sp	堆栈指针 (Stack Pointer)
\$30	\$fp	帧指针 (Frame Pointer)
\$31	\$ra	函数返回地址 (Return Address)

图5.25 add指令各字段十进制值

• (2) I型指令（立即数型）

I 型指令

6bits

OP

5bits

R_s

5bits

R_t

16bits

立即数

- I型指令就是**立即数型**指令
- 若是双目运算，则将源操作数 rs 和立即数 imm 分别作为第一和第二操作数，结果送 rt
- 若是Load/Store指令，则将 rs 的值和立即数 imm 的值相加，生成存储单元地址
- 若是条件分支指令，则对 rs 和 rt 的值进行比较，并根据比较结果决定是否转移，转移的**目标地址采用相对寻址方式**
- 常见的I型指令及其操作码如表5.9所示

将字地址转成字节地址
 $imm * 4$

表5.9 常用I型指令及其操作码

OP	指令助记符	指令功能描述	备注
04	beq rs, rt, imm	If($R[rs] == R[rt]$) $PC = PC + 4 + imm \ll 2$	条件分支（相等跳转） imm 左移2bit，末位00，确保地址为4的倍数，因为字长是32bit=4Byte
05	bne rs, rt, imm	If($R[rs] != R[rt]$) $PC = PC + 4 + imm \ll 2$	条件分支（不等跳转）
08	addi rt, rs, imm	$R[rt] = R[rs] + imm$	立即数加，溢出发生异常
10	slti rt, rs, imm	$R[rt] = (R[rs] < imm) ? 1 : 0$	小于置位指令，有符号比较
12	andi rt, rs, imm	$R[rt] = R[rs] \& imm$	立即数逻辑与指令
15	lui rt, imm	$R[rt] = imm \ll 16$	加载立即数指令，长度对齐，左移16bit，正好是32bit
35	lw $rt, imm(rs)$	$R[rt] = M[R[rs] + imm]$	取数指令，类似指令还有lb、lh、lbh等
43	sw $rt, imm(rs)$	$M[R[rs] + imm] = R[rt]$	存数指令，类似指令还有sb、sh

• (3) J型指令（无条件转移型）

J 型指令

6bits

OP

26bits

立即数

address

- J型指令主要是**无条件转移**指令
- 指令中给出的是**26位直接地址（address）**，采用**伪直接寻址方式**；**无条件转移的目标地址由PC+4的高4位(4bit)address经左移2位后（28bit）的值拼接得到，即为32bit.**
- 例如：PC+4=2500 0000H，address=1234567H，PC+4的高4位=2H，address左移2位=01 0010 0011 0100 0101 0110 0111 00= 0100 1000 1101 0001 0101 1001 1100=48D159CH，拼接后的目标地址=248D 159CH
- 常用的J型指令及其操作码如表5.10所示

表5.10 常用J型指令及其操作码

OP	指令助记符	指令功能描述	备注
02	j address	PC <- {(PC+4) _{31:28} ,address,00}	无条件分支
03	jal address	R[31] <- PC+8 (无延迟槽+4) PC <- {(PC+4) _{31:28} ,address,00}	子程序调用指令

存放断点，即返回地址
有延迟槽+8；延迟槽已经流水线中执行了
无延迟槽+4；尚未执行
注：延迟槽为跳转指令后面的那一条或多条指令

改进方案

为了提高流水线的效率，避免气泡，MIPS的架构做了两个改进：

- 1、分支指令的判断提前到**译码阶段**，一部分比较逻辑在译码时完成，而不用等到执行阶段。这样可以少一个气泡。
- 2、索性不清空取指阶段指令，让它继续执行，反正下一个指令是跳转指令，只要有标记知道它是“多出”的指令，称为“**延迟槽指令**”。所以把转移指令后面的指令位置称为“**延迟槽**”。这样流水线就没有气泡了。

5.7.3 MIPS指令系统

— 3、MIPS指令寻址方式

- MIPS指令只有5种寻址方式，如表5.11所示
- R型指令：只有寄存器寻址
- I型指令：有寄存器寻址、立即数寻址、基址寻址（寄存器相对寻址）、相对寻址
- J型指令：只有伪直接寻址

表5.11 MIPS指令的寻址方式

序号	寻址方式	有效地址EA/操作数S	指令示例
1	立即数寻址	$S = \text{imm}$	<code>addi rt,rs,imm</code>
2	寄存器寻址	$S = R[\text{rt}]$	<code>add rd,rs,rt</code>
3	基址寻址（寄存器相对寻址）	$EA = R[\text{rs}] + \text{imm}$	<code>lw rt,imm(rs)</code>
4	相对寻址	$EA = PC + 4 + \text{Disp}$	<code>beq rs,rt,imm</code>
5	伪直接寻址	$EA = \{(\text{PC} + 4)_{31:28}, \text{address}, 00\}$	<code>j address</code>

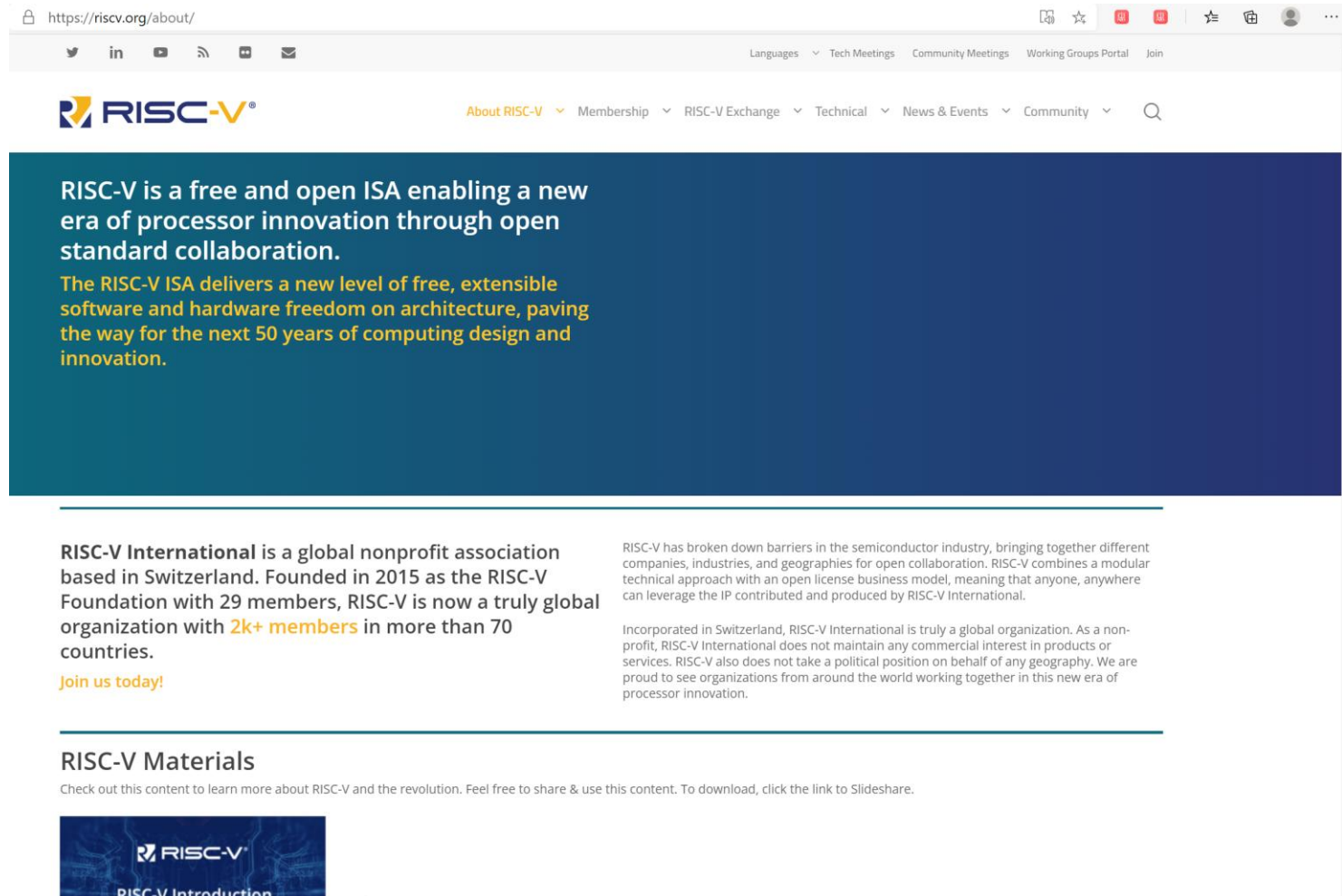


无条件转移的目标地址由PC+4的高4位(4bit)address经左移2位后(28bit)的值拼接得到，即为32bit

5.7.4 RISC-V指令系统

- RISC-V（发音：RISC Five）是**第五代RISC指令系统**，其结合了ARM、MIPS等RISC指令系统的优势，是完全从零开始重新设计的一款开源指令系统，由RISC-V基金会负责运营
- RISC-V指令集采用**模块化**的方式设计，包括**32位指令集**和**64位指令集**
- **32位RISC-V指令集（RV32G）**包括**核心指令集RV32I**，以及**4个标准扩展集：RV32M（乘除法）、RV32F（单精度浮点数）、RV32D（双精度浮点数）、RV32A（原子操作）**
- **核心指令集RV32I**：只有**47条指令**，其指令格式规整，易于硬件的实现
- **RISC-V模块化的指令集设计方法**，方便进行灵活定制与扩展，既可以用于嵌入式微处理器（MCU），也适合构造服务器、家用电器、工业控制以及传感器中的微处理器（CPU）
- 阿里巴巴旗下的**平头哥**半导体有限公司的**玄铁处理器**就采用了**RISC-V架构**

RISC-V基金会（<https://riscv.org/>）



The image is a screenshot of the RISC-V website's 'About' page. The browser's address bar shows 'https://riscv.org/about/'. The website's header includes the RISC-V logo, a navigation menu with links like 'About RISC-V', 'Membership', 'RISC-V Exchange', 'Technical', 'News & Events', and 'Community', and a search icon. The main content area has a dark blue background with white and yellow text. It states that RISC-V is a free and open ISA enabling processor innovation through open standard collaboration. It also mentions that the RISC-V ISA delivers a new level of free, extensible software and hardware freedom on architecture, paving the way for the next 50 years of computing design and innovation. Below this, there is a section titled 'RISC-V International' which describes it as a global nonprofit association based in Switzerland, founded in 2015 as the RISC-V Foundation with 29 members, and now a truly global organization with 2k+ members in more than 70 countries. A 'Join us today!' link is provided. To the right, there is a paragraph explaining that RISC-V has broken down barriers in the semiconductor industry, bringing together different companies, industries, and geographies for open collaboration. Below this, another paragraph states that RISC-V International is truly a global organization, does not maintain any commercial interest in products or services, and does not take a political position on behalf of any geography. At the bottom, there is a section titled 'RISC-V Materials' with a link to check out content to learn more about RISC-V and the revolution, and a link to download content. A small video player thumbnail for 'RISC-V Introduction' is visible at the bottom left.

<https://riscv.org/about/>

RISC-V

About RISC-V Membership RISC-V Exchange Technical News & Events Community

RISC-V is a free and open ISA enabling a new era of processor innovation through open standard collaboration.

The RISC-V ISA delivers a new level of free, extensible software and hardware freedom on architecture, paving the way for the next 50 years of computing design and innovation.

RISC-V International is a global nonprofit association based in Switzerland. Founded in 2015 as the RISC-V Foundation with 29 members, RISC-V is now a truly global organization with **2k+ members** in more than 70 countries.

[Join us today!](#)

RISC-V has broken down barriers in the semiconductor industry, bringing together different companies, industries, and geographies for open collaboration. RISC-V combines a modular technical approach with an open license business model, meaning that anyone, anywhere can leverage the IP contributed and produced by RISC-V International.

Incorporated in Switzerland, RISC-V International is truly a global organization. As a non-profit, RISC-V International does not maintain any commercial interest in products or services. RISC-V also does not take a political position on behalf of any geography. We are proud to see organizations from around the world working together in this new era of processor innovation.

RISC-V Materials

Check out this content to learn more about RISC-V and the revolution. Feel free to share & use this content. To download, click the link to Slideshare.

RISC-V

RISC-V Introduction

平头哥和玄铁

- **平头哥**半导体有限公司于**2018年10月31日**成立，法定代表人刘湘雯，是阿里巴巴旗下半导体公司。**2019年9月25日**，阿里巴巴在**2019云栖大会**上正式对外发布含光**800AI**芯片。平头哥半导体将打造面向汽车、家电、工业等诸多行业领域的物联网芯片平台。前期由阿里扶持，后期平头哥将会独立化运作，自负盈亏。
- **玄铁910**是平头哥发布的基于**RISC-V**的处理器IP核，开发者可以免费下载**FPGA**代码，开展芯片原型设计架构创新。**2019年7月25日**，平头哥首个产品玄铁910正式发布。



5.7.4 RISC-V指令系统

– 1、RISC-V通用寄存器

- 32个32位的通用寄存器：x0 ~ x31；也可以表示为：zero、ra、.....、t6；见表5.12

表5.12 RISC-V的通用寄存器

编号	助记符	英文全称	功能描述
x0	zero	zero	恒零值，可用0号寄存器参与的加法指令实现MOV指令
x1	ra	Return Address	返回地址
x2	sp	Stack Pointer	栈指针，指向栈顶
x3	gp	Global Pointer	全局指针
x4	tp	Thread Pointer	线程寄存器
x5 ~ x7	t0 ~ t2	Temporaries	临时变量，调用者保存寄存器
x8	s0/fp	Saved Register/Frame Pointer	通用寄存器，被调用者保存寄存器，在子程序中使用时必须先压栈保存原值，使用后应出栈恢复原值
x9	s1	Saved Registers	通用寄存器，被调用者保存寄存器
x10 ~ x11	a0 ~ a1	Arguments/Return values	用于存储子程序参数或返回值
x12 ~ x17	a2 ~ a7	Arguments	用于存储子程序参数
x18 ~ x27	s2 ~ s11	Saved Registers	通用寄存器，被调用者保存寄存器
x28 ~ x31	t3 ~ t6	Temporaries	临时变量

– 2、RISC-V指令格式

- [ir指令、立即数的作用_我们一起学RISC-V](#)
- RISC-V也是**定长指令**（32位），但其操作码字段预留了扩展空间，可以扩展为变长指令，但指令**字长必须是双字节对齐**
- RISC-V有**6种指令格式**，如图5.26所示
- 操作码**opcode=7位**，**funct3（3位）**、**funct7（7位）**为扩展操作码；**rs1、rs2**是源操作数寄存器，**rd**是目的操作数寄存器，都是**5位**，用于选择**32个寄存器**，且位置固定；其余位置用于填充立即数字段，因此有些格式的指令（**S型、B型、J型**）立即数字段分散在几个地方
- RISC-V也没有寻址方式字段，**寻址方式由操作码决定**；RISC-V指令字中各字段位置固定，可以提高指令译码的速度，**硬件实现也更容易**

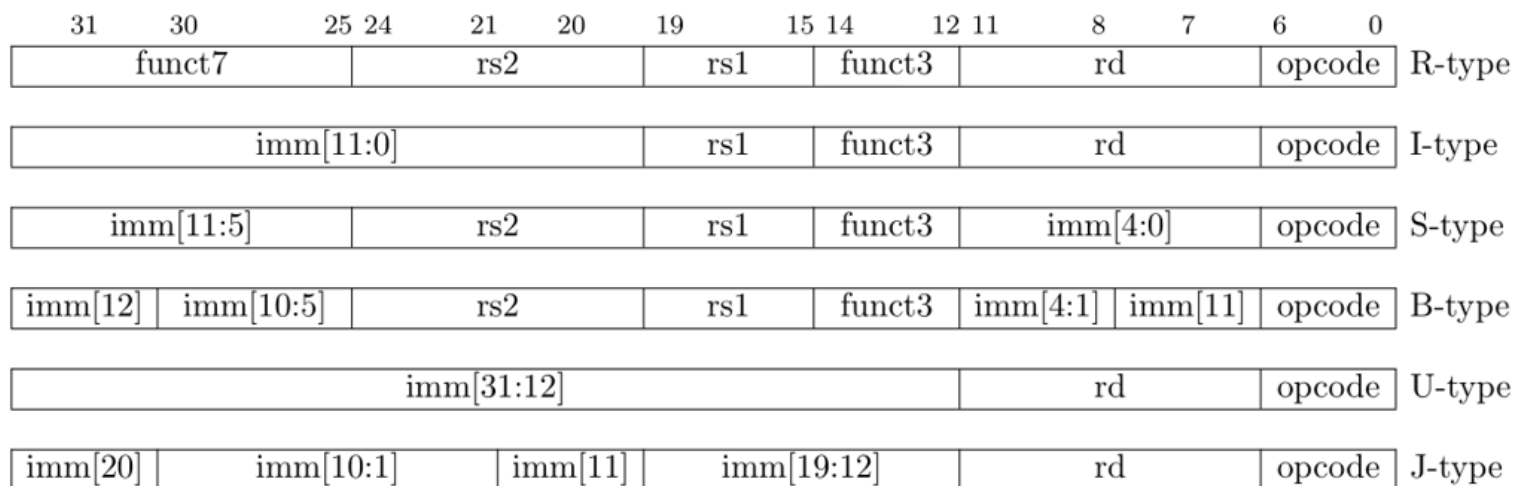


图5.26 RISC-V的指令格式

5.7.4 RISC-V指令系统

- (1) R型指令（寄存器型）
 - R型指令的操作数只能是寄存器（rs1、rs2），运算结果也只能存入寄存器（rd），属于**RR（寄存器-寄存器）型**指令
 - 主操作码**OP=33H**，指令的功能由**funct3（3位）**、**funct7（7位）**字段指定，如图5.27所示
 - 核心指令集RV32I中有**10条R型指令**，如表5.13所示



图5.27 R型指令

表5.13 RISC-V的R型指令

类型	指令示例	功能描述	同类指令
算术逻辑运算指令	add rd,rs1,rs2	$R[rd]=R[rs1]+R[rs2]$	add、sub、xor、or、and
关系运算指令	slt rd,rs1,rs2	$R[rd]=(R[rs1]<R[rs2])?1:0$	slt、sltu
移位指令	sll rd,rs1,rs2	$R[rd]=R[rs1]<<R[rs2]$	sll、srl、sra

5.7.4 RISC-V指令系统

- (2) I型指令（立即数型）

- I型（立即数型）指令的操作数为两个寄存器（rs1、rd）和立即数（12位）
- 操作码包括主操作码OP（7位）以及扩展操作码funct3（3位），如图5.28所示
- 核心指令集RV32I中有25条R型指令，如表5.14所示

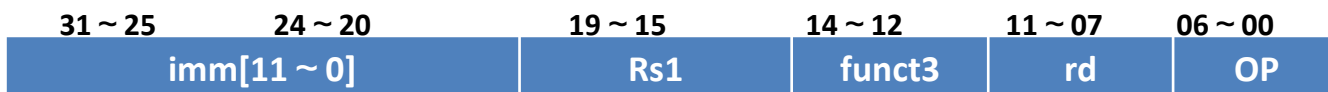


图5.28 I型指令

表5.14 RISC-V的I型指令

跳转到新的PC，同时将返回地址，即断点存入目的寄存器

访存指令包含在I型指令中

类型	指令示例	功能描述	同类指令
算术逻辑运算指令	addi rd,rs1,imm	$R[rd]=R[rs1] + imm$	addi、xori、ori、andi
关系运算指令	slti rd,rs1,imm	$R[rd]=R[rs1] < imm$	slti、sltiu
移位指令	slli rd,rs1,imm	$R[rd]=R[rs1] \ll imm$	slli、srli、srai
访存指令	lw rd,imm(rs1)	$R[rd]=M[R[rs1] + imm]$	lb、lbu、lh、lhu、lw
系统控制类指令	jalc rd,rs1,imm	$PC=R[rs1] + imm$ $R[rd]=PC+4$	jalc
系统控制类指令	ecall	系统调用	ecall、fence、fence.l、ebreak
特权指令	csrrw rd,csr,rs1	$R[rd]=csr$ $csr=R[rs1]$	csrrw、csrrs、csrrc、csrrwi、csrrsi、csrrci

5.7.4 RISC-V指令系统

- (3) S型指令（写存储器型）

- 写存储器型指令由于不存在目的寄存器rd，因此不能使用I型指令的格式，只能单独设置一个S型指令格式；其操作数包括两个寄存器rs1、rs2，以及立即数imm（两部分，共12位）
- 操作码包括主操作码OP（7位）以及扩展操作码funct3（3位），如图5.29所示
- 核心指令集RV32I中有3条S型指令，如表5.15所示

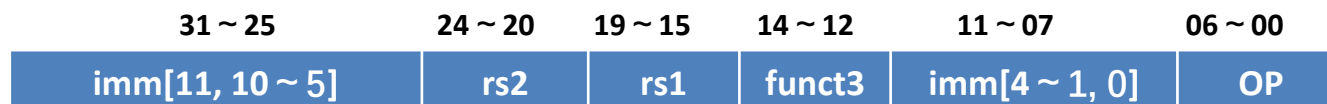


图5.29 S型指令

表5.15 RISC-V的S型指令

类型	指令示例	功能描述	同类指令
访存指令	sw rs2,imm(rs1)	$M[R[rs1] + imm] = R[rs2]$	sw、sb、sh

5.7.4 RISC-V指令系统

- (4) B型指令（条件分支型）
 - B型指令用于表示**条件分支指令**，B型指令也不存在目的寄存器rd，其指令格式与S型指令类似；其操作数包括两个寄存器rs1、rs2，以及立即数imm（两部分，共12位，与S型指令略有不同）；B型指令也称为**SB型指令**
 - 操作码包括主操作码OP（7位）以及扩展操作码funct3（3位），如图5.30所示
 - 核心指令集RV32I中有**6条B型指令**，如表5.16所示

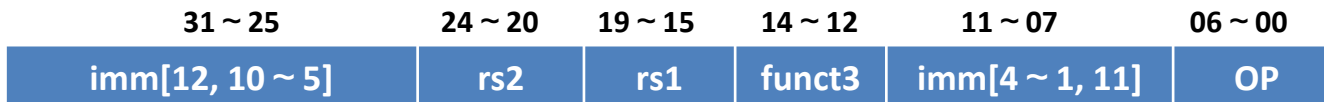


图5.30 B型指令

RISC-V规定指令字长为双字节的倍数，所以左移后，地址末位置0，保证其为偶数。

表5.16 RISC-V的B型指令

类型	指令示例	功能描述	同类指令
分支指令	beq rs1,rs2,imm	if([R[rs1] == R[rs2]]) PC=PC+imm<<1	beq、bne、blt、bge、bltu、bgeu

5.7.4 RISC-V指令系统

- (5) U型指令（更大立即数型）
 - I型指令的立即数只有12位，范围较小；U型指令的立即数有20位，范围较大；U型指令只有一个目的寄存器rd
 - 操作码只有主操作码OP（7位），如图5.31所示
 - 核心指令集RV32I中有2条U型指令，如表5.17所示



图5.31 U型指令

表5.17 RISC-V的U型指令

类型	指令示例	功能描述	注释
立即数加载	lui rd,imm	$R[rd] = \text{imm} \ll 12$	加载有符号立即数到高20位，即imm到rd的高20bit，imm取值范围为0-1048575
	lui a0,0x23 // 此时a0的数值为0x23000		
立即数加载	auipc rd,imm	$R[rd] = PC + \text{imm} \ll 12$	加载有符号立即数imm的到PC寄存器高20位，同时保存结果到rd中
	auipc a1, 0x06 // 假设当前pc数值为0x04，执行该指令后，a1的数值为0x6004		

5.7.4 RISC-V指令系统

- (6) J型指令（无条件跳转型）
 - J型指令用于实现**无条件跳转**，其操作数包括一个目的寄存器rd和20位立即数；J型指令与U类指令格式类似，也称为**UJ型指令**
 - 操作码只有主操作码OP（7位），如图5.32所示
 - 核心指令集RV32I中有**1条J型指令**，如表5.18所示

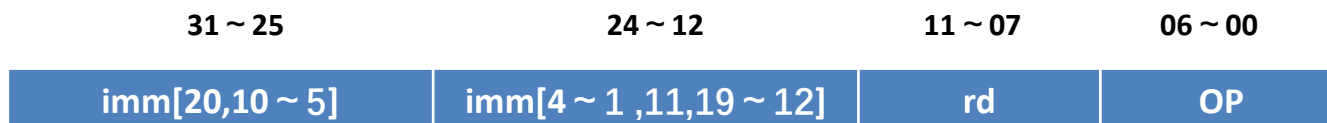


图5.32 J型指令

表5.18 RISC-V的J型指令

类型	指令示例	功能描述
子程序调用	jal rd,imm	$PC = PC + imm \ll 1$ $R[rd] \leftarrow PC + 4$ rd为x1时可实现子程序调用；rd=x0时，可实现无条件跳转

$Imm \ll 1$ ，表示将字地址转换成字节地址。因为RISC规定地址是**双字节对齐**。**Rd**用于存放子程序返回地址

- **RISC-V核心指令集RV32I的47条指令：**
 - 移位指令（6条）：**sll、slli、srl、srli、sra、srai**
 - 算术运算指令（5条）：**add、addi、sub、lui、adipc**
 - 逻辑运算指令（6条）：**xor、xori、or、ori、and、andi**
 - 比较指令（4条）：**slt、slti、sltu、sltiu**
 - 分支指令（条件转移指令）（6条）：**beq、bne、blt、bge、bltu、bgeu**
 - 跳转指令（无条件转移指令）（2条）：**jal、jalr**
 - 同步指令（2条）：**fence、fence.i**
 - 环境指令（2条）：**ecall、ebreak**
 - 控制状态寄存器读写指令（6条）：**csrrw、csrrs、csrrc、csrrwi、csrrsi、csrrci**
 - 取数指令（5条）：**lb、lh、lbu、lhu、lw**
 - 存数指令（3条）：**sb、sh、sw**

5.7.4 RISC-V指令系统

— 3、RISC-V寻址方式

- RISC-V比MIPS少了伪直接寻址方式，只有4种寻址方式，如表5.19所示

表5.19 RISC-V指令的寻址方式

序号	寻址方式	有效地址EA/操作数S	指令示例
1	立即数寻址	$S = \text{imm}$	<code>addi rd,rs1,imm</code>
2	寄存器寻址	$S = R[\text{rs1}]$	<code>add rd,rs1,rs2</code>
3	基址寻址（寄存器相对寻址）	$EA = R[\text{rs1}] + \text{imm}$	<code>lw rd,imm(rs1)</code>
4	相对寻址	$EA = PC + \text{imm} \ll 1$	<code>beq rs1,rs2,imm</code>

5.7.5 ARMv7指令系统（补充）

— ARM: Advanced RISC Machines

- ARM公司1991年成立于英国剑桥，32位嵌入式RISC微处理器业界的领先IP核供应商（IP核：Intellectual Property，知识产权核或知识产权模块），ARM公司本身不生产芯片，而是通过转让设计方案由合作伙伴生产各具特色的芯片。

— ARMv7: ARM系列处理器的架构（体系结构）版本7（最新版本ARMv9）

— ARMv7处理器有3个家族：

- **Cortex-A**系列又称高性能处理器（**Highest Performance**），它是面向移动计算如智能手机、平板电脑和服务器市场定制的高端处理器内核，支持了包括Linux、Android、Windows和iOS等系统必须的内存管理单元（MMU），而且也是与我们平时接触最为密切的存在。**Cortex-A**系列面向尖端的基于虚拟内存的操作系统和用户应用。
- **Cortex-R**系列实时处理器（**Real-Time Processing**）为要求可靠性、高可用性、容错功能、可维护性和实时响应的嵌入式系统提供高性能计算解决方案。**Cortex-R**系列处理器通过已经在数以亿计的产品中得到验证的成熟技术提供极快的上市速度，并利用广泛的ARM生态系统、全球和本地语言以及全天候的支持服务，保证快速、低风险的产品开发。**Cortex-R**系列针对实时系统。
- **Cortex-M**系列针对成本和功耗敏感（**Lowest Power, Lower Cost**）的MCU和终端应用（如智能测量、人机接口设备、汽车和工业控制系统、大型家用电器、消费性产品和医疗器械）的混合信号设备进行过优化。**Cortex-M**系列针对微控制器。

— 大多数ARM处理器支持两种或两种以上指令集：

- **ARM指令集**：32位
- **Thumb指令集**：16位、32位，是ARM指令集的一个子集
- **ThumbEE指令集**：是Thumb指令集的一个变体

ARM公司

- **ARM**公司是一家知识产权（**IP**）供应商，它与一般的半导体公司最大的不同就是不制造芯片且不向终端用户出售芯片，而是通过转让设计方案，由合作伙伴生产出各具特色的芯片。**ARM**公司利用这种双赢的伙伴关系迅速成为了全球性**RISC**微处理器标准的缔造者。这种模式也给用户带来巨大的好处，因为用户只掌握一种**ARM**内核结构及其开发手段，就能够使用多家公司相同**ARM**内核的芯片。



5.7.5 ARMv7指令系统（补充）

– 1、ARMv7寄存器

- 包括通用寄存器和程序状态寄存器

- 通用寄存器：

- » 未备份寄存器：**R0-R7**
 - » 备份寄存器（影子寄存器）：**R8-R14**（**R13**: **SP**，堆栈指针；**R14**: **LR**，链接寄存器）
 - » 程序计数器**PC**: **R15**

- 程序状态寄存器：

- » **CPSR**: 当前程序状态寄存器
 - » **SPSR**: 备份程序状态寄存器

CPSR: Current Program Status Register, 当前程序状态寄存器
SPSR: Saved Program Status Register, 备份程序状态寄存器

- 寄存器数量：

- 通用寄存器：9种工作模式共34个
 - 程序状态寄存器：9种工作模式共8个

5.7.5 ARMv7指令系统（补充）

- **ARMv7支持9种工作模式：**

- ① **User**：用户进程运行在 **User** 模式下，拥有受限的系统访问权限
- ② **System (Sys)**：系统模式，这个模式下将与用户模式共享寄存器视图
- ③ **FIQ**：快中断异常处理模式，相对于中断而言，快中断拥有更高的响应等级、更低延迟。
- ④ **IRQ**：中断异常处理模式
- ⑤ **Abort (ABT)**：内存访问异常处理模式，常见的 **MMU fault** 就会跳转到该模式下进行相应的处理
- ⑥ **Supervisor (SVC)**：内核通常运行在该模式下，在系统复位的时候或者应用程序调用 **svc** 指令的时候将会进入到当前模式下，系统调用就是通过 **svc** 指令完成
- ⑦ **Undefined (UND)**：当执行未定义的指令时，触发硬件异常，硬件上自动跳转到该模式下
- ⑧ **Monitor (MON)**：针对安全扩展，在该模式下执行 **secure** 和 **non-secure** 处理器状态的切换
- ⑨ **Hypervisor (HYP)**：针对虚拟化扩展

- **用户态和内核态：**

- 用户态的工作模式：**User**；
- 内核态（核态）的工作模式：**Sys、FIQ、IRQ、ABT、SVC、UND、MON、HYP**

ARMv7处理器不同模式下的可访问寄存器

R0	R0	R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7	R7	R7
R8	R8	R8_fiq	R8	R8	R8	R8	R8	R8
R9	R9	R9_fiq	R9	R9	R9	R9	R9	R9
R10	R10	R10_fiq	R10	R10	R10	R10	R10	R10
R11	R11	R11_fiq	R11	R11	R11	R11	R11	R11
R12	R12	R12_fiq	R12	R12	R12	R12	R12	R12
R13 (sp)	R13 (sp)	SP_fiq	SP_svc	SP_abt	SP_svc	SP_und	SP_mon	SP_hyp
R14 (lr)	R14 (lr)	LR_fiq	LR_svc	LR_abt	LR_svc	LR_und	LR_mon	
R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)

共34个通用寄存器

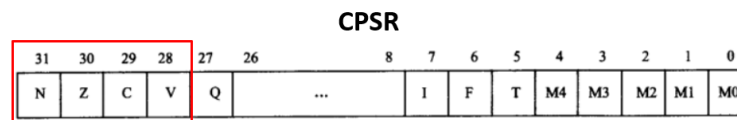
(A/C)PSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_fiq	SPSR_irq	SPSR_abt	SPSR_svc	SPSR_und	SPSR_mon	SPSR_hyp
								ELR_hyp
User	Sys	FIQ	IRQ	ABT	SVC	UND	MON	HYP

共8个程序状态寄存器

5.7.5 ARMv7指令系统（补充）

— 2、ARMv7指令格式

- 指令格式：<opcode> {<cond>} {S} <Rd>, <Rn> {, <shift_op2>}
- <>内的项是必须的，{}内的项是可选的
- opcode: 指令助记符（操作码），如LDR, STR等
- **cond**: 执行条件（条件域/条件码），如EQ, NE等，可以没有
- S: 可选后缀（可以没有），加S时影响CPSR中的条件码标志位（NZCV），不加S时则不影响
- Rd: 目标寄存器
- Rn: 第1个源操作数的寄存器
- op2: 第2个源操作数，可以没有
- **shift**: 位移操作，可以没有



条件码标志位

- 条件码（**cond**）：CPSR[31:28] N Z C V
 - N（**负数标志位**）：运算结果的b31位值。对于有符号二进制补码，结果为负数时N=1，结果为正数或零时N=0
 - Z（**零标志**）：指令结果为0时Z=1，否则Z=0
 - C（**进位标志**）：使用加法运算（包括CMN指令），b31位产生进位时C=1，否则C=0。使用减法运算（包括CMP），b31位产生借位时C=0，否则C=1。对于结合移位操作的非加法 / 减法指令，C为b31位最后的移出值，其它指令C通常不变
 - V（**溢出标志**）：使用加法 / 减法运算，当发生有符号溢出时V=1，否则V=0，其它指令V通常不变
- 位移操作（**shift**）：
 - LSL: 逻辑左移
 - LSR: 逻辑右移
 - ASR: 算术右移
 - ROR: 循环右移
 - RRX: 带扩展的循环右移（加上CPSR的C位一起循环右移）

条件码（**NZCV**）的16种组合

AMR指令条件码	助记符	描述	CPSR条件码标志位的值
0000	EQ	相等，运行结果为0	Z置位
0001	NE	不相等，运行结果不为0	Z清零
0010	CS/HS	无符号数大于等于	C置位
0011	CC/LO	无符号数小于	C清零
0100	MI	负数	N置位
0101	PL	非负数	N清零
0110	VS	上溢出	V置位
0111	VC	没有上溢出	V清零
1000	HI	无符号数大于	C置位且Z清零
1001	LS	无符号数小于等于	C清零且Z置位
1010	GE	带符号数大于等于	N=V
1011	LT	带符号数小于	N!=V
1100	GT	带符号数大于	Z清零且N=V
1101	LE	带符号数小于等于	Z置位且N!=V
1110	AL	无条件执行	
1111	系统保留		

位移操作<shift_op2>的11种形式

语法	含义
#<immediate>	立即数寻址
<Rm>	寄存器寻址
<Rm>, LSL #<shift_imm>	立即数逻辑左移
<Rm>, LSL <Rs>	寄存器逻辑左移
<Rm>, LSR #<shift_imm>	立即数逻辑右移
<Rm>, LSR <Rs>	寄存器逻辑右移
<Rm>, ASR #<shift_imm>	立即数算术右移
<Rm>, ASR <Rs>	寄存器算术右移
<Rm>, ROR #<shift_imm>	立即数循环右移
<Rm>, ROR <Rs>	寄存器循环右移
<Rm>, RRX	寄存器扩展循环右移

5.7.5 ARMv7指令系统（补充）

– 3、ARMv7寻址方式（9种）

- （1）立即寻址

- **ADD R1, R1, #0x1** ; R1+1 -> R1

- » #：立即寻址的前缀

- » #0x：十六进制

- » #0b：二进制

- » #0d：十进制

- » ARM指令通常有3个操作数：1个目的操作数，2个源操作数

- （2）寄存器寻址

- **ADD R1, R1, R2** ; R1+R2 -> R1

- **MOV R1, R0** ; R0 -> R1

- （3）寄存器移位寻址（寄存器偏移寻址）

- **ADD R1, R1, R2, ROR #0x2** ; R2循环右移（**ROR**）2位后与R1相加，结果放入R1

- **MOV R1, R0, LSL R2** ; R0逻辑左移（**LSL**）R2位后 -> R1

5.7.5 ARMv7指令系统（补充）

- （4）寄存器间接寻址
 - STR R1, [R2] ; R1 -> [R2], 即将R1存入[R2]存储单元中
 - SWP R1, R1, [R2] ; [R2] 和 R1 的内容进行交换
 - STR: 存数指令
 - SWP: 交换指令
- （5）基址寻址（基址变址寻址）
 - 基址寻址有4种形式:
 - ①零偏移（Zero Offset）
 - op Rd, [Rn, R1] ; 有效地址 = $Rn + R1$
 - ②前索引偏移（Pre-Indexed）
 - op Rd, [Rn, FlexOffset] ; 有效地址 = $Rn + FlexOffset$
; FlexOffset: 称为灵活的偏移量
 - ③带写回的前索引偏移（Pre-Indexed with Writeback）
 - op Rd, [Rn, FlexOffset]! ; 有效地址 = $Rn + FlexOffset$, 并且 $Rn + FlexOffset \rightarrow Rn$
; “!” 表示自增
 - ④后索引偏移（Post-Indexed）
 - op Rd, [Rn], FlexOffset ; 有效地址 = Rn , 并且 $Rn + FlexOffset \rightarrow Rn$

5.7.5 ARMv7指令系统（补充）

- （6）多寄存器寻址（块数据寻址）
 - **LDMIA** R0, {R1, R2, R3, R4, R5} ; [R0] -> R1, [R0+4] -> R2, [R0+8] -> R3, [R0+12] -> R4, [R0+16] -> R5
 - **STMIA** R0, {R2-R5, R7} ; R2 -> [R0], R3 -> [R0+4], R4 -> [R0+8], R5 -> [R0+12], R7 -> [R0+16]
 - **LDM**: 批量加载（读取）指令，连续从一片存储器读数，送一组寄存器中
 - **STM**: 批量存储指令，将一组寄存器中的数，连续存入一片存储器中
 - **IA**（后增）：操作完成后，地址递增
 - **IB**（先增）：地址先增，而后完成操作
 - **DA**（后减）：操作完成后，地址递减
 - **DB**（先减）：地址先减，而后完成操作
 - **I**: 增加（增），**Increase**
 - **D**: 减少（减），**Decrease**
 - **A**: 之后（后），**After**
 - **B**: 之前（先），**Before**

5.7.5 ARMv7指令系统（补充）

- （7）堆栈寻址

- 堆栈指针（SP）：R13

- 满堆栈：Full Stack

Empty Stack

空堆栈：

- 递增堆栈：Ascending Stack

Descending Stack

递减堆栈：

- 满递增堆栈（满增）：FA 满递减堆栈（满减）

: FD

- 空递增堆栈（空增）：EA 空递减堆栈（空减）

: ED

- STMFD SP!, {R1-R7, LR}

; 将LR、R7-R1，存放到堆栈中，

; LR -> [SP-4], R7 -> [SP-8], R6 -> [SP-12],

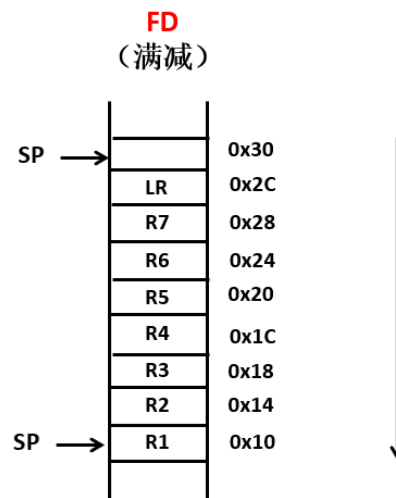
; R5 -> [SP-16], R4 -> [SP-20], R3 -> [SP-24],

; R2 -> [SP-28], R1 -> [SP-32],

; “!” 表示自减（FD）

开始时：SP=0x30

结束时：SP=0x10



• (8) 相对寻址

- 相对寻址是基址寻址的一个特例，PC作为基地址，一般用于指令跳转

BL Label ; 转到Label标签处

...

Label:

...

- **BL指令**：带链接的跳转指令

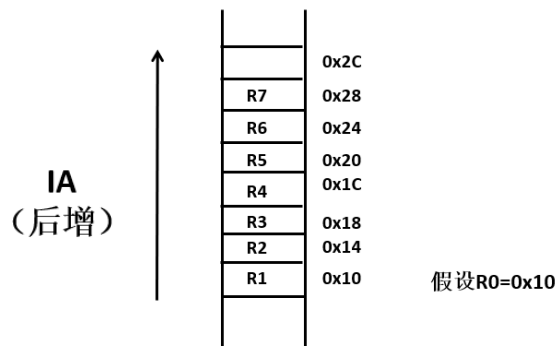
• (9) 块复制寻址（块拷贝寻址）

- **STMIA R0!, {R1-R7}** ;将R1~R7的数据保存到[R0]开始的存储器中，存储指针在保存第一个值之后递增
- **STMIB R0!, {R1-R7}** ;将R1~R7的数据保存到[R0]开始的存储器中，存储指针在保存第一个值之前递增
- **STMDA R0!, {R1-R7}** ;将R7~R1的数据保存到[R0]开始的存储器中，存储指针在保存第一个值之后递减
- **STMDB R0!, {R1-R7}** ;将R7~R1的数据保存到[R0]开始的存储器中，存储指针在保存第一个值之前递减

- **IA（后增）**：操作完成后，地址递增；
- **IB（先增）**：地址先增，而后完成操作
- **DA（后减）**：操作完成后，地址递减
- **DB（先减）**：地址先减，而后完成操作

- **I**：增加（增），Increase
- **D**：减少（减），Decrease
- **A**：之后（后），After
- **B**：之前（先），Before

- **!**：表示自增（自减）



STMIA R0!, {R1-R7}

5.7.5 ARMv7指令系统（补充）

— 4、ARMv7常用指令（26条）

- （1）数据处理指令（16条）

① **MOV**: 传送指令 **MOV** R0, R1 ; R1 -> R0

② **MVN**: 取反后传送指令 **MVN** R0, R1 ; /R1 -> R0

» 指令格式: **op {cond} {S} Rd, Operand2** ; Operand2 -> Rd

» **cond**: 执行条件, 如EQ, NE 等

» **S**: 可选后缀, 有S时将影响CPSR寄存器的值, 否则不影响

— 例子:

» **MOV** R0, R0, **ASR** R2 ; R0算术右移 (**ASR**) R2位 -> R0

» **MVN****NE** R1, #0x22 ; 不相等 (**NE**) 时, /0x22 -> R1 (
0xFFFFFFFFDD -> R1)

5.7.5 ARMv7指令系统（补充）

- ③ **ADD**: 加法指令 **ADD R1,R2,R3 ; R2+R3 -> R1**
- ④ **ADC**: 带进位加法指令 **ADC R1,R2,R3 ; R2+R3+C -> R1**
- ⑤ **SUB**: 减法指令 **SUB R1,R2,R3 ; R2-R3 -> R1**
- ⑥ **SBC**: 带进位减法指令 **SBC R1,R2,R3 ; R2-R3+C-1 -> R1**
- ⑦ **RSB**: 逆向减法指令 **RSB R1,R2,R3 ; R3-R2 -> R1**
- ⑧ **RSC**: 带进位逆向减法指令 **RSC R1,R2,R3 ; R3-R2+C-1 -> R1**

— 指令格式: **op {cond} {S} Rd, Rn, Operand2**

» **cond**: 执行条件, 如EQ, NE 等

» **S**: 可选后缀, 有**S**时将影响CPSR寄存器的值, 否则不影响

— 例子:

» **ADDS R0, R1, #1280 ; 带S后缀, R1 + #1280 -> R0**

» **SUBHI R1, R2, R3 ; 如果“无符号数大于 (HI)”, R2-R3 -> R1**

5.7.5 ARMv7指令系统（补充）

⑨ **AND**: 逻辑与 **AND R1,R2,R3 ; R2&R3 -> R1**

⑩ **ORR**: 逻辑或 **ORR R1,R2,R3 ; R2|R3 -> R1**

⑪ **EOR**: 逻辑异或 **EOR R1,R2,R3 ; R2⊕R3 -> R1**

⑫ **BIC**: 位清零 **BIC R1,R2,R3 ; R2+/R3 -> R1**

– 指令格式: **op {cond} {S} Rd, Rn, Operand2**

» **cond**: 执行条件, 如EQ, NE 等

» **S**: 可选后缀, 有S时将影响CPSR寄存器的值, 否则不影响

– 例子:

» **EOR R0, R1, #xFF00 ; R1 逻辑异或 #xFF00 -> R0**

» **BIC R0, R1, R1 ; R1 逻辑与 /R1 -> R0, 相当于 0 -> R0**

» **ORR R1, R2, R4, LSR #2 ; R2 逻辑或 R4逻辑右移 (LSR) 2位 -> R1**

» **BICNES R5, R6, R1, RRX ; 带S后缀, 如果不相等 (NE), 则R6逻辑与
; “R1带扩展循环右移 (RRX) 后取反” -> R5**

5.7.5 ARMv7指令系统（补充）

⑬ **TST**: 位测试指令

» **TST R1, R2** ;R1和R2按位进行**逻辑与**操作

⑭ **TEQ**: 相等测试指令

» **TEQ R1, R2** ;R1和R2按位进行**逻辑异或**操作

⑮ **CMP**: 比较指令

» **CMP R1, R2** ;R1和R2**相减**

⑯ **CMN**: 反值比较指令

» **CMN R1, R2** ;R1和R2**相加**

— 指令格式: **op {cond} Rn, Operand2**

» **cond**: 执行条件, 如EQ, NE 等

— 例子:

» **TST R1, #0x0F** ; R1 和 #0x0F 按位进行**逻辑与**操作

» **TEQ**NE** R9, #0x4000** ; 如果不相等 (**NE**),
; 则: R19 和 0x4000 按位进行**逻辑异或**操作

5.7.5 ARMv7指令系统（补充）

- （2）分支跳转指令（3条）

- ① **B**: 基本的跳转指令

- » 指令格式: **B {cond}, Label** ;(PC) + Label -> PC

- ② **BL**: 带链接的跳转指令（子程序调用指令）

- » 指令格式: **BL {cond}, Label** ;PC -> LR, (PC) + Label -> PC

- ③ **BX**: 子程序返回指令

- » 指令格式: **BX {cond}, Rm** ;Rm -> PC

– 例子:

..... ; 主程序

BL Label ; 子程序调用

.....

Label: ; 子程序的第一条指令

.....

BX LR ; 子程序返回

5.7.5 ARMv7指令系统（补充）

- （3）状态寄存器访问指令（2条）

- ① **MRS**: 状态寄存器（CPSR、SPSR）读指令

- 指令格式: **MRS {cond} Rd, psr** ; psr -> Rd
 - psr: CPSR或SPSR

- ② **MSR**: 状态寄存器（CPSR、SPSR）写指令

- 指令格式: **MSR {cond} <psr>_<field>, #immed_8r** ; #immed_8r -> psr
 - psr: CPSR或SPSR
 - 指令格式: **MSR {cond} <psr>_<field>, Rm** ; Rm -> psr
 - psr: CPSR或SPSR

- 例子:

- **MRS R5, SPSR** ; SPSR -> R5
 - **MSR CPSR_f, R7** ; R7->CPSR_f, 即更新状态寄存器的标志位

5.7.5 ARMv7指令系统（补充）

- （4）访存指令（5条）

- ① **LDR**: 从存储器读数，送寄存器
- ② **STR**: 将寄存器中的数，存入存储器
- ③ **LDM**: 批量加载（连续从一片存储器读数，送一组寄存器中）
- ④ **STM**: 批量存储（将一组寄存器中的数，连续存入一片存储器中）
- ⑤ **SWP**: 交换指令，用于寄存器和存储器之间的内容交换

– 例子:

- **LDR** R0, [R1] ; [R1]存储单元的内容 -> R0
- **LDRSH** R0, [R1, #0xF2] ; [R1 + #0xF2]存储单元内容的带符号（**S**）半字（**H**） -> R0
- **STRH** R2, [R0, R3]! ; [R0 + R3]存储单元内容的无符号半字（**H**） -> R2, 并且 R0+R3 -> R0
- **LDRB** R4, Lable ; [Label]存储单元的无符号字节（**B**） -> R4
- **LDMFA** R5, {R1,R3,R5,R7} ; 将[R5]开始的4个存储单元的内容 -> R1、R3、R5、R79（**FA** : 满增）
- **STMFD** R13!, {R0-R4,LR} ; 将R0-R4及LR，压入堆栈（R13即SP）（**FD**: 满减）
- **LDMFD** R13!, {R0-R4,PC} ; 从堆栈中读（R13即SP），送到R0-R4及PC（**FD**: 满减）
- **SWPB** R2, R3, [R4] ; 字节（**B**）交换: [R4] -> R2, R3 -> [R4]
- **SWP** R1, R1, [R5] ; [R5] -> R1, R1 -> [R5], 即R1和[R5]的内容交换
- **PLD** [R2, #Label *5] ; 有效地址=R2+ Label *5
- **PLD** [R3, R2, **LSR** #0x2] ; 有效地址=R3 + R2逻辑右移（**LSR**）2位

实践训练（实验3）

- 在**MIPS**汇编器**MARS**中，利用最少的**MIPS**指令编写一个内存数据冒泡排序程序
- 在**RISC-V**汇编器**RARS**中，利用最少的**RISC-V**指令编写一个内存数据冒泡排序程序